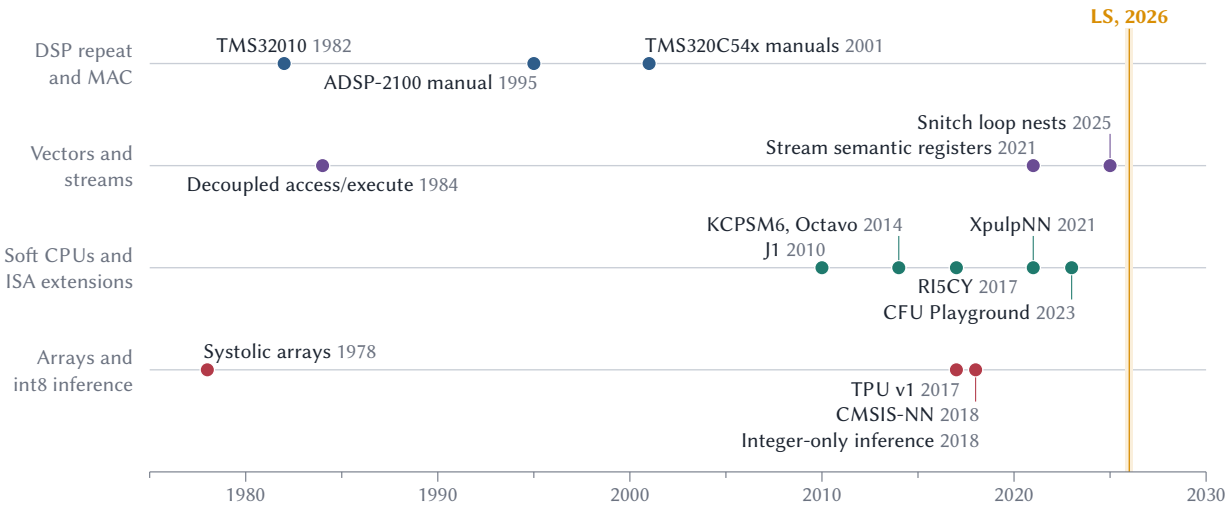


Prior art behind the LS design, in four lanes. DSP years are those of the manuals cited; the amber band marks LS (2026).

The LS Architecture

State of the art

Small processors, int8 multiply-accumulate, and what it takes to keep one multiplier busy

Loucas Louka · Stefanos Bordea

105 verified references · searches and verification 8–9 November 2026

Findings in brief.

1. The multiplier is cheap. An int8 dot product is paid for in instruction fetch and decode, operand delivery and loop control (Horowitz, 2014; Dally et al., 2020).
2. The remedy is the DSP recipe: a single-cycle MAC into a wide accumulator, two operand reads per cycle, post-incrementing address generators and zero-overhead loops. DSPs have had it since the 1980s; RISC-V and Arm microcontroller extensions have repeated it since 2017.
3. Bit-serial, compute-in-BRAM, dataflow and EDGE designs carry overheads that do not pay off for one neuron on an FPGA with 240 hard DSP slices.
4. The LS design adopts the hardware repeat as `MAC #n`. It passes the review's three-part structural test, and the predicted 64-input neuron takes 112 cycles against 300 for the best conventional code on the same core.
5. No mechanism is claimed as new. What is uncommon is the combination at 8-bit scale and its honest measurement on the board.

Contents

1 Summary 1

2 Scope and search method 1

3 Framing theory: why the workload decides the architecture 3

3.1 The iron law of performance 3
3.2 Roofline and the operand bound 3
3.3 Data-movement and instruction energy . 4
3.4 The von Neumann bottleneck 4
3.5 Domain-specific architectures 4

4 Thematic review 5

4.1 Minimal soft CPUs 5
4.2 ISA extensions for quantized neural networks 7
4.3 DSP mechanisms 8
4.4 Alternative execution models 9
4.5 Neural-network accelerators: systolic, bit-serial, binary 10
4.6 FPGA fabric and compute-in-BRAM . . . 12
4.7 Quantization and requantization 13
4.8 TinyML benchmarks and datasets 14
4.9 Across the clusters 15

5 Comparison matrix 16

6 Closest comparators and what is uncommon at LS scale 19

7 Implications for the LS design decisions 20

8 Must-cite shortlist for the report 24

9 Limitations of this review 24

References 26

Reading note

This review was written against the project overview of 8 November 2026, and its *For LS* boxes refer to that plan. The design document (spec v0.1, 9 November 2026) adopted the review's main recommendation, a hardware repeat that streams two operands per clock into one MAC (`MAC #n`), and fixed the instruction set; Sections 1, 3.1, 6 and 7 use its cycle counts. Numbers marked *derived* are calculations, not measurements. Citations follow APA style.

1 Summary

This review asked which execution model makes an 8-bit FPGA CPU structurally efficient at int8 multiply-accumulate (MAC) work, what the state of the art (SOTA) is, and what is uncommon at the LS project's scale; it rests on 105 verified references. The multiplier is cheap: an int8 dot product costs instruction fetches, operand delivery and loop control (Horowitz, 2014; Dally et al., 2020; Kung, 1982), and most speed-up comes from the parallelism that specialization enables (Dally et al., 2020). The mechanisms that remove the overhead (a single-cycle MAC into a wide accumulator, two operand reads per cycle, post-incrementing address generators, zero-overhead loops) have prior art in 1980s–1990s DSPs (Eyre & Bier, 2000; Texas Instruments, 2001b) and in 32-bit RISC-V and Arm microcontroller extensions from 2017 to 2026 (Gautschi et al., 2017; Yiu, 2019; Nadalini et al., 2023). The Artix-7's block-RAM ports reach 36 bits, or 72 in simple dual-port mode (Xilinx, 2019), so one MAC per cycle suits the 8-bit instruction set and the December demo rather than marking a fabric limit. Bit-serial, compute-in-BRAM, dataflow and EDGE models carry overheads that do not pay off for one neuron. With the instruction set of the LS design document (spec v0.1, four FSM cycles per instruction), a 64-input neuron is predicted at 1,328 cycles without a MAC instruction, 560 with a MAC loop, 300 with 64 unrolled MACs and 112 under the hardware repeat `MAC #n` (derived; Section 3.1). Candidate prior art exists for 8-bit cores with MAC hardware and for integer-only inference on 8-bit microcontrollers, so neither is claimed as new. What remains defensible is a combination and its measurement: an 8-bit soft core on Artix-7 that streams int8 operands from two operand memories into a DSP48E1 MAC (Xilinx, 2018) under a hardware repeat, with an int8 requantize, measured on the board against the best conventional code for the same core, and bit-exact against a reference model.

2 Scope and search method

Scope. The target is the design in the LS project overview (internal team document: docs/project-overview.md in the team's 8-bit-cpu repository, version of 8 November 2026): 8-bit data, four registers (R0–R3), 16-bit instructions, 12-bit addresses, pointers P0 and P1 that the MAC instruction advances, a 24-bit accumulator, a four-step FSM (fetch, decode, execute, write back), a one-instruction-per-press step mode, and a Nexys A7-100T (XC7A100T-1) at 100 MHz. Course rules come from the CSEN 203L semester-project brief (course document). Neither document is literature, so neither is in the reference list.

Sources. Metadata came from Crossref, arXiv, Semantic Scholar, OpenAlex, DataCite, Europe PMC, CiNii and Open Library; texts and numbers from publisher and proceedings sites (including the publisher's companion site for the online Section 4.5 of Patterson & Hennessy, 2021), vendor portals, author repositories, the GitHub API and Wayback Machine snapshots. dblp, IEEE Xplore full text, the ACM Digital Library and ScienceDirect were blocked; Google Scholar was not queried.

Cluster	Condensed queries
Minimal soft CPUs	J1 Forth CPU EuroForth 2010; KCPSM6 PicoBlaze 7-series; SERV, PicoRV32, VexRiscv area and Fmax; FazyRV datapath width; soft-core custom instruction int8 MAC TinyML Artix-7 2024–2025
ISA extensions	RISC-V DSP extensions (Gautschi); XpulpNN; PULP-NN; CMSIS-NN; RISC-V P extension status; Zvqdotq; integrated matrix extension; Armv8.1-M low-overhead branch
DSP mechanisms	TMS320C54x RPT, RPTZ, MAC; ADSP-2100 DO UNTIL; zero-overhead loop controller; overhead-free FPGA soft processor; stream semantic registers; hardware loop RISC-V 2024–2025
Execution models	transport-triggered soft cores; BrainTTA; decoupled access/execute HLS 2023–2026; Manchester dataflow; tagged-token dataflow; Pipestitch; triggered instructions FPGA; TRIPS evaluation; EDGE soft processor; ELI-512; stack computers
Accelerators	TPU v1; DianNao; Eyeriss; Stripes; Pragmatic; Loom; Bit Fusion; UNPU; BISMO; binarized networks; XNOR-Net; FINN; precision-scalable MAC benchmark; bit-serial 2024 HPCA/MICRO
FPGA fabric	UG479, UG473, DS180, DS181 (–1 grade); WP486 INT8; Double MAC DSP48E1; compute-in-BRAM (CCB, CoMeFa, BRAMAC, M4BRAM); PiCaSO; IMAGine; direct BRAM-DSP paths
Quantization	integer-arithmetic-only inference; LiteRT quantization specification; accumulator-aware quantization (A2Q, A2Q+, AXE); accumulator-constrained processors; dyadic quantization; power-of-two thresholds
TinyML benchmarks	MLPerf Tiny v1.3; MLPerf Power; TinyML benchmarking challenges; UCI optical digits; MNIST; Ethos-U55; hls4ml and FINN on Arty A7-100T; TinyML progress survey
Follow-up check (9 Nov 2026): 8-bit cores and KB-sized ML	8-bit microcontroller hardware MAC and wide accumulator; AVR or 8051 ISA extension with MAC; compiling KB-sized ML models to microcontrollers; CORE-V-MCU on Nexys A7; MAXQ transport-triggered microcontroller

Inclusion criteria. A source was kept if it dealt with int8 dot-product efficiency in small programmable hardware, FPGA primitives, quantization arithmetic or benchmarking; if its authors, title, venue and year matched a DOI record, publisher page, arXiv record or official vendor document; and if every number used here was read in the source (abstract-only reading is flagged). Peer-reviewed work was preferred, vendor documents supply device facts, and preprints, documentation and drafts are labelled. Any doubt about metadata meant a drop. Work from 2023–2026 was searched for specifically.

Dates and counts. Searches ran on 8 November 2026, the follow-up check on 9 November, and verification on 8–9 November. Of 116 entries that reached verification, 111 were kept and 5 dropped for metadata mismatches. The DSP48E1 guide (Xilinx, 2018) was submitted by three clusters, and the memory guide (Xilinx, 2019), Bowman (2010), Gautschi et al. (2017) and Prakash et al. (2023) by two each; removing these six duplicates leaves 105 unique references: 66 journal, conference or magazine articles, 6 books or chapters, 12 vendor documents, 10 preprints and 11 other gray sources (documentation, drafts, a non-refereed paper, a report, a dataset record, a blog post), 30 of them from 2023–2026. The 5 dropped entries are four works, since two clusters submitted the MLPerf Tiny paper; the others were the NEORV32 datasheet (its DOI resolves to a software release), the Motorola DSP56000 manual (date mismatch) and the TensorFlow Lite Micro paper (author-list mismatch). About 45 candidates were screened out earlier. Leads from the follow-up check are listed in Section 9; they are not counted and serve only to narrow Sections 1 and 6. Numbers computed for this review are marked “derived”.

3 Framing theory: why the workload decides the architecture

3.1 The iron law of performance

Execution time is instructions × cycles per instruction (CPI) × clock period. Harris and Harris (2021, Sections 7.3–7.5) work it through for 100 billion instructions:

Design	CPI	Clock period	Time	Speed vs single-cycle
Single-cycle	1	750 ps	75 s	1
Multicycle (FSM)	4.12	375 ps	155 s	0.5
Pipelined	1.23	350 ps	43 s	1.7

Patterson and Hennessy (2021, Section 4.5) assume that a multicycle clock cycle holds at most one memory access, one register-file access or one ALU operation (p. 282.e1); their example gives CPI 4.10 for a SPECINT2006 mix (p. 282.e12), against 4.12 for the SPECINT2000 mix in Harris and Harris. A multicycle FSM saves hardware, not time. For LS the same arithmetic predicts each program of the design document (spec v0.1), which keeps the overview's four FSM cycles per instruction and adds a fused decrement-and-branch (`DJNZ`) and post-increment loads:

Rung	Inner loop, per MAC	Instructions per MAC	Predicted cycles per MAC	64-input neuron, whole program (N = 32)
B: no MAC instruction	`LD`, `LD`, `MUL`, `ADDP`, `DJNZ`	5	20	1,328 (688)
L: MAC in a loop	`MAC`, `DJNZ` (the MAC advances both pointers)	2	8	560 (304)
U: MAC unrolled	`MAC` written 64 times	1	4	300 (172)
S: hardware repeat	`MAC #64`: the FSM fetches and decodes once, then streams	0 in steady state	1	112 (80)

Every program also spends 44 to 48 cycles on set-up and output; `MAC #n` itself takes n + 4 cycles, so the fixed cost c of the overview's plan is 4 cycles plus the program's set-up. All entries are derived from the listings in the design document and confirmed on a cycle-accurate model of the spec. The fair comparison is S against U, the best code the same core can run without the repeat: 300 / 112 = 2.68× for 64 inputs, and one cycle per input against four. The B and L rungs show where the first gains come from; quoting B / S would credit the repeat with the gain of having a MAC instruction at all (Section 4.2).

3.2 Roofline and the operand bound

Attainable throughput is the lesser of peak compute and memory bandwidth × operational intensity (Williams et al., 2009). Kung (1982) gave the special-purpose version: at two bytes per operation, a 10 MB/s host link caps a system at 5 million operations per second "no matter how fast the special-purpose system can operate", while a six-cell systolic array on the same memory reaches 30 MOPS through operand reuse.

For LS (derived): each MAC needs a weight byte and an input byte. At batch size one each weight is used once, but each input feeds every neuron of the layer, 32 times in a 64-32-10 network, so L lanes sharing one broadcast input byte need L weight bytes plus one input byte per cycle. A RAMB36 port can be 36

bits wide (four data bytes) in true dual-port mode or 72 bits in simple dual-port mode, each port with its own width (Xilinx, 2019), and the XC7A100T has 135 RAMB36 (Xilinx, 2020). One weight port can feed four lanes while the other stays byte-wide, so the one-MAC-per-cycle bound comes from the design's byte-wide operand memories, not from the fabric. It fits an 8-bit instruction set, one accumulator and a December demo of one neuron, which has no second output to share an input with; lanes are the structural next step for the digit network (Section 7).

3.3 Data-movement and instruction energy

Horowitz's (2014) 45 nm figures put arithmetic at the bottom of the energy bill:

Operation (45 nm, 0.9 V)	Energy
8-bit integer add / 32-bit integer add	0.03 pJ / 0.1 pJ
8-bit integer multiply / 32-bit integer multiply	0.2 pJ / 3.1 pJ
32-bit floating-point multiply	3.7 pJ
64-bit cache read: 8 KB / 32 KB / 1 MB	10 / 20 / 100 pJ
DRAM access	1.3–2.6 nJ
One instruction on a simple in-order processor	about 70 pJ (instruction cache 25 pJ, register file 6 pJ)

An int8 multiply plus a 32-bit add costs about 0.3 pJ, roughly 1/230 of the energy of the instruction that carries it (derived). Eyeriss gives the same ordering relative to one MAC: DRAM 200×, global buffer 6×, neighbouring processing element 2×, register file 1× (Y.-H. Chen et al., 2016). Dally et al. (2020) report that fetching and interpreting an instruction costs 10× to 4000× as much as an ADD, that a simple in-order core spends over 90% of its energy on overhead, and that one Turing IMMA instruction performs 1,024 integer operations with about 87% of its energy in arithmetic. Their first key insight, however, is that "most speedup comes from parallelism enabled by specialization", with specialization as the main source of efficiency. They also put FPGA logic 10–100× below ASIC efficiency, so these ASIC figures do not transfer directly (Dally et al., 2020). On the board, the only energy figure available without a meter is Vivado's power estimate (Section 7).

3.4 The von Neumann bottleneck

Backus (1978) described "a connecting tube that can transmit a single word between the CPU and the store", much of whose traffic is "not useful data but merely names of data". His running example is the inner product c := c + a[i] × b[i], with loop "housekeeping" spread through the program. That example is the LS workload.

3.5 Domain-specific architectures

Hennessy and Patterson (2019) chart processor performance growth slowing to an "End of the Line" of "2X/20 years (3%/yr)" and argue that domain-specific architectures win through more efficient parallelism, software-controlled memory, lower precision ("inference regularly uses 4-, 8-, or 16-bit integers") and domain-specific languages; TPU v1 was 29× faster and over 80× more energy-efficient than a CPU. Dally et al. (2020) add that "arithmetic is free and global memory is expensive" and that specialized instructions keep much of a DSA's advantage while staying programmable.

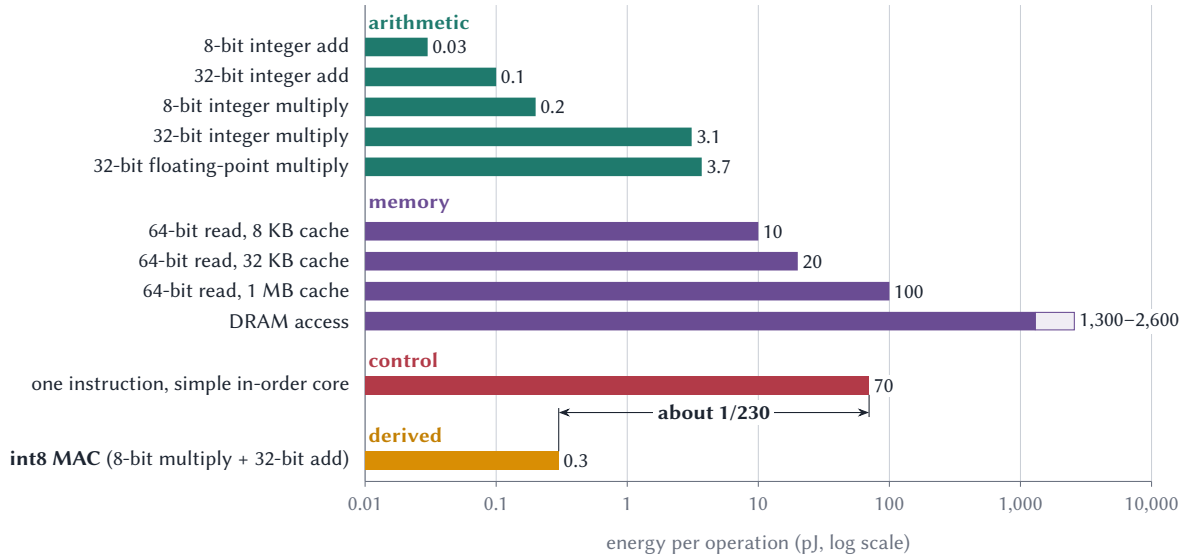


Figure 1. Energy per operation at 45 nm and 0.9 V (Horowitz, 2014), on a logarithmic scale; the pale segment marks the 1.3–2.6 nJ range of a DRAM access. An int8 multiply-accumulate (8-bit multiply plus 32-bit add) costs about 0.3 pJ (derived), roughly 1/230 of one instruction on a simple in-order core (about 70 pJ).

What "structural" means in this review. A design is structural for the dot product when its inner loop (S1) fetches and decodes no instruction per MAC, (S2) receives every operand a MAC needs in the cycle it is needed (two bytes for one lane, L + 1 for L broadcast lanes), and (S3) keeps the partial sum in the MAC unit until requantization. Passing all three keeps the multipliers busy every cycle by construction; further speed then comes from lanes where operands are reused (Dally et al., 2020). Section 7 scores all eight candidates against S1–S3.

4 Thematic review

4.1 Minimal soft CPUs

Core	Style	Area	Clock	Cycles
SERV (Kindgren, 2018)	Bit-serial RV32I	125 LUT, 164 FF on Artix-7 (register file excluded)	not stated	32 per one-stage instruction, plus fetch
FazyRV (Kissich & Baunach, 2024)	RV32I, 1–8-bit chunks	example SoC in 77 slices (7-series)	not extracted	addition: 32 cycles at 1 bit, 4 at 8 bits
KCPSM6 (Chapman, 2014)	8-bit, 18-bit instructions	26 slices + 1 BRAM (Spartan-6)	238 MHz Kintex-7 (–3); no Artix-7 row	2 for every instruction; no multiply
PicoRV32 (Wolf, 2015)	Multicycle state machine	761–2,019 LUT (Kintex-7, synthesis only)	416–454 MHz Kintex/Virtex-7	about 4 on average; MUL 40
VexRiscv (Papon, 2017)	2–5+ stage pipeline	504–1,418 LUT (Artix-7, fastest grade)	243–216 MHz	0.52–1.21 DMIPS/MHz
GRVI (Gray, 2016)	Hand-mapped RV32I	320 LUT	375 MHz Kintex UltraScale	CPI estimated 1.3–1.6
FemtoRV32 (Levy & Koch, 2020)	Teaching state machine	980–1,180 LUT with SoC (iCE40)	80 MHz (iCE40)	mostly 2–3

Core	Style	Area	Clock	Cycles
J1 (Bowman, 2010)	16-bit dual stack; one dual-port RAM for code and data	not reported	80 MHz (Spartan-3E)	frequent Forth words 1; about 100 ANS Forth MIPS
RI5CY (Gautschi et al., 2017)	4-stage RV32 with two hardware-loop levels, post-increment, SIMD dot product; open RTL	extensions +6.6 kGE	65/28 nm ASIC only	+37% average from loops and post-increment

State of the art. Bit-serial cores set the 32-bit area floor at 32 or more cycles per instruction (Kindgren, 2018), and FazyRV is fastest at its 8-bit setting (Kissich & Baunach, 2024). KCPSM6 is the 8-bit reference, with CPI 2 and no multiply (Chapman, 2014). PicoRV32, a non-pipelined state machine with CPI near 4, is the closest match to the LS control plan (Wolf, 2015); among seven open cores tested in hardware it was among the fastest in MHz but had the lowest relative performance (Heinz et al., 2019). J1 runs frequent Forth words in one cycle because the ALU works in parallel with fetch and decode and the stacks are register files (Bowman, 2010). GRVI is laid out for the 6-input LUT (Gray, 2016). RI5CY's RTL is open source, but the verified sources report it only as an ASIC (Gautschi et al., 2017).

CFU Playground pairs a VexRiscv with a custom function unit (CFU) on small FPGA boards (Prakash et al., 2023). On an Arty A7-35T (XC7A35T, DDR3, 100 MHz system clock), int8 MobileNetV2's 1×1 convolutions sped up cumulatively 9.8× with a four-way MAC instruction, 26× (under one cycle per MAC) once the inner loop and operand reads moved into the unit, and 55× with post-processing and overlap; the whole model gained only 3×. On a Fomu board (iCE40UP5k, 12 MHz, code and weights in SPI flash), keyword spotting gained 75×, of which only 3× came from the CFU; memory-system, CPU and software changes supplied the rest, including a single-cycle multiplier in place of an iterative ~30-cycle one (Prakash et al., 2023). Flex-RV couples a SIMD MAC unit to a SERV-derived core through four instructions (two multiply modes, bias with non-linearity, requantize); it runs a 4-bit model, reports no speed-up over software, and passes every operand through the register file (Ozer et al., 2024).

Reporting cautions. Processing-element figures in Heinz et al. (2019, Table II: PicoRV32 about 4,040–4,250 LUTs and VexRiscv about 3,780–3,910 LUTs on four Xilinx boards, including controller and local memory) are roughly 2–3× the core-only figures of Wolf (2015; 2,019 LUTs, large configuration, Kintex-7 synthesis) and Papon (2017; 1,418 LUTs, full no-cache configuration, fastest-grade Artix-7) (derived; configurations and devices differ). SERV's area excludes its register file. No core in the table reports post-route Fmax on an Artix-7 –1 part.

> **For LS**
>
> The planned FSM is the textbook machine, so the efficiency claim cannot rest on it. CFU Playground shows where the structural gain comes from: cycles per MAC fell below one when the loop and the operand fetch left the instruction stream. Use the textbook multicycle CPU as the internal baseline and KCPSM6 as the external one, and report core and system resources separately.

4.2 ISA extensions for quantized neural networks

Mechanism and baseline (same core without it unless stated)	Measured effect	Source
Hardware loops + post-increment, compiler C	+37% average; convolutions up to +41%	Gautschi et al. (2017)
Post-increment alone	up to +20% with regular access	Gautschi et al. (2017)
Adding SIMD dot-product built-ins	3.5× average, up to 13.2× on data-intensive kernels	Gautschi et al. (2017)
int8 kernel library with sdotp4 vs plain RV32IMC	8.8× (INT-8 convolution, one core); 3.69× at INT-4	Garofalo, Rusci, et al. (2020)
Operand load fused into the MAC, vs PULP-NN kernel	OPEF 0.57 ⟶ 0.80–0.94; 1.39–1.65×	Garofalo et al. (2021)
4-bit and 2-bit SIMD vs 8-bit SIMD only	5.3× and 8.9× on convolutions	Garofalo, Tagliavini, et al. (2020)
Custom 4-way MAC, then whole loop in the unit (Arty A7-35T, MobileNetV2 1×1 convolutions)	9.8×, then 26× (< 1 cycle per MAC), cumulative	Prakash et al. (2023)
Packed SIMD vs the same core's RV32IM (post-implementation simulation)	up to 7× on matrix multiplication, 5.96× on LeNet-5	Ali et al. (2025)
Vector unit vs a cacheless MicroBlaze (modelled)	25–33× on dot products, 1.4–1.9× on 2-D convolution, blamed on scalar pointer management	Al Assir et al. (2021)
Removing an unused FPU from a vector lane	most of a 43.3% lane-area saving	Dupuis et al. (2023)
256-byte result buffer next to the arithmetic	vector-register-file power −53.5% for under 3% area	Perotti et al. (2024)

State of the art. Three mechanisms recur in microcontroller-class instruction sets: zero-overhead hardware loops (about 1.5 kGE per loop level in RI5CY), post-increment addressing, and a packed dot product adding four int8 products into a 32-bit accumulator (Gautschi et al., 2017; Garofalo, Rusci, et al., 2020; RISC-V International, 2026a). Armv8.1-M added low-overhead loops that cache the loop addresses on the first pass (Yiu, 2019). Armv7E-M has no int8 MAC, so CMSIS-NN widens int8 data to 16 bits before a dual 16-bit MAC (Lai et al., 2018). The evidence is concentrated: eight papers in this section and the next (Gautschi et al., 2017; Garofalo, Rusci, et al., 2020; Garofalo, Tagliavini, et al., 2020; Garofalo et al., 2021; Schuiki et al., 2021; Nadalini et al., 2023; Perotti et al., 2024; Colagrande et al., 2025) share a senior author and measure gains on that group's own cores against its own baselines. Independent support comes from Arm (Yiu, 2019), the TI and ADI manuals (Texas Instruments, 2001b; Analog Devices, 1995), ZOLC (Kavvadias & Nikolaidis, 2008) and Octavo on an FPGA (LaForest et al., 2014).

With those mechanisms in place, loads become the limit. XpulpNN's MAC operation efficiency (OPEF), the share of executed instructions that are MACs, is 0.33 with two loads per MAC and no reuse, 0.57 for the register-blocked PULP-NN kernel, and 0.80–0.94 once the operand load and address update are folded into the dot-product instruction, for a 17.5% larger core (Garofalo et al., 2021). The FPGA studies in the table point the same way but use weaker baselines.

Standards (November 2026). None of these RISC-V drafts is ratified: the integrated matrix extension, with int8-to-int32 operations, is a v0.9.0 review candidate (RISC-V International, 2026b); the P draft, with the byte dot product PM4ADDA.B, is at v0.6 (RISC-V International, 2026a); and the non-saturating vector

dot product Zvqdotq v0.0.2 (RISC-V International, 2025) is being replaced by a renamed v0.7 draft, so cite version and date.

For LS

The plan has two of the three mechanisms, auto-advancing pointers and a wide-accumulator MAC, but no hardware loop. Packed int8 SIMD needs 32-bit registers; independent MAC lanes fed from a wide weight port do not (Section 3.2). The accumulator belongs in the MAC unit, not in R0–R3. Baselines need a hardware multiplier: on the Fomu board, CFU Playground's cumulative keyword-spotting speed-up had already reached 15.35× at the step that replaced the iterative multiply (Prakash et al., 2023), so a baseline without one inflates every later gain.

4.3 DSP mechanisms

System	Mechanism	Cycle budget
TMS320C54x (Texas Instruments, 2001a, 2001b)	`MAC *AR5+, *AR6+` reads two operands and post-increments both pointers; RPTZ clears the accumulator and repeats the next instruction	MAC 1 cycle; RPTZ 2; RPT #K 1; block repeat 4 (2 delayed); N-term dot product N + 2 (derived)
ADSP-2100 (Analog Devices, 1995)	One instruction does the MAC and fetches the next two operands from data and program memory; DO UNTIL loop	FIR listing: N − 1 + 5 + 2 cycles, i.e. N + 6 (derived)
ZOLC (Kavvadias & Nikolaidis, 2008)	Zero-overhead loop controller for a RISC or ASIP	MAC loop 10 ⟶ 6 cycles; 298–4,428 gates
Octavo (LaForest et al., 2014)	Address-offset and branch-trigger modules in an FPGA soft CPU	1.07–1.92× across benchmarks; 8-tap FIR 1.16× looping, 1.09× unrolled; Fmax × 0.939
Stream semantic registers (Schuiki et al., 2021)	Registers mapped to address-generator streams, plus hardware loop	dot-product utilization 17% ⟶ 100% (analytic); +11% core area
Flex-V (Nadalini et al., 2023)	MAC fused with load; 2-D address generator with stride, rollback, skip	int8: 26.9 MAC/cycle on 8 cores (XpulpNN 26.1, RI5CY 16.6)
Snitch with loop nests (Colagrande et al., 2025)	Nested zero-overhead loops, conflict-free memory	FPU utilization 85% ⟶ 96.1–99.4%; < 3% area

State of the art. The recipe dates from the TMS32010 of 1982, the first commercially successful DSP with a single-cycle multiply. One MAC per cycle means fetching the instruction, a sample and a coefficient in the same cycle, which DSPs do with a single-cycle MAC into a wide accumulator with guard bits, two operand fetches per cycle from separate memories, post-incrementing and circular address generators, and zero-overhead loops (Eyre & Bier, 2000; Lapsley et al., 1997). Such "multiple memory spaces, multiple buses, irregular instruction sets, and highly specialized hardware, are difficult for compilers to use effectively", so critical code is often hand-optimized in assembly (Eyre & Bier, 2000, BDTI white-paper version). Both manuals keep eight guard bits above the product: C54x accumulator bits 39–32 are "headmargin" (Texas Instruments, 2001a), and on the ADSP-2100 "256 overflows would have to occur before any data is lost" (Analog Devices, 1995). Repeated 3-cycle C54x MAC variants "effectively become single-cycle instructions after the first iteration", and interrupts are disabled during a repeat (Texas Instruments, 2001a).

The table's gains need context. ZOLC cut average cycles by 25.5% on RISC kernels and 44% on an ASIP, and its 40% MAC-loop example assumes a 3-cycle branch penalty (Kavvadias & Nikolaidis, 2008). Oc-

tavo's FIR kept its eight taps and coefficients in registers (LaForest et al., 2014, Table I). Stream semantic registers reach 20% utilization with hardware loops alone (Schuiki et al., 2021). Flex-V moves 2-D pointer management, about 30% of instructions by the authors' estimate, into hardware and keeps MAC utilization above 80%; its headline 91.5 MAC/cycle is a 2-bit result (Nadalini et al., 2023). Even with streams, outer-loop management adds an asymptotic overhead of 2/(K × unroll) (Colagrande et al., 2025).

For LS

The planned MAC with auto-advancing pointers follows the C54x and ADSP-2100 pattern, minus the repeat and the two reads per cycle. Three additions make it structural (Section 3.5):

- a repeat or hardware loop, so the FSM fetches and decodes once and then streams;
- inputs and weights on the two independent block-RAM ports (Xilinx, 2019);
- optionally, a second loop level that resets the input pointer for each neuron, as Flex-V's stride and rollback registers do.

A 24-bit accumulator over 16-bit products keeps the DSPs' eight guard bits. A hardware repeat conflicts with one-instruction-per-press step mode only if the repeat hides inside one instruction; Section 7 makes the repeat count part of the FSM state, so one press advances one MAC.

4.4 Alternative execution models

Model	Evidence	Key numbers	Costs reported
Transport-triggered (TTA)	Jääskeläinen et al. (2018), 32-bit cores on Zynq Z7020	single-issue: 956 LUT at 216 MHz vs MicroBlaze-3 715 LUT at 169 MHz, register file 24 vs 128 LUT, 15–173% faster than MicroBlaze-5; dual-issue: 67% of a VLIW's resources, up to 88% faster	in that design: 43-bit instructions, 265-LUT interconnect, programs 21–49% larger than VLIW, cycles not lower on every program
TTA for NN inference	Molendijk et al. (2023), 22 nm	2.47 TOPS/W at 8 bits, 28.6 binary	ASIC; abstract only
VLIW	Fisher (1983); Jääskeläinen et al. (2018)	4-read, 2-write register file 638 LUT vs 44 LUT for a 1-read, 1-write TTA file	instruction width
Decoupled access/execute (DAE)	Smith (1984)	1.1–2.1×, average 1.58, on 14 Livermore loops (simulation)	needs latency to hide
DAE in HLS	Metz & Själander (2026)	decoupling alone 1.58–40.57× vs Vitis, 100-cycle memory	not 1-cycle BRAM
Tagged-token dataflow	Gurd et al. (1985); Arvind & Nikhil (1990)	1–2 MIPS; duplicate instructions 25% of those executed; associative matching	matching bottleneck
Ordered dataflow	Serafin et al. (2023)	3.49× over RipTide at 1.10× area (threaded apps)	ASIC fabric
Triggered instructions	Parashar et al. (2013)	2.0× vs program-counter spatial baseline; 8× area-normalized vs a general-purpose processor	no small FPGA implementation among verified sources
EDGE	Gebhart et al. (2009); Gray & Smith (2018)	moves ~20% of instructions; code ~11× Alpha; 32-entry scheduler 150–340 LUT	code size
Stack	Koopman (1989); Bowman (2010)	32-deep stack suffices; J1 code about 62% smaller than the equivalent C code on a MicroBlaze-type core	J1 implementation: no multiplier, one dual-port RAM shared by code and data

State of the art. In a TTA (transport-triggered architecture) the program moves data between function-unit ports, and an operation fires when an operand reaches a trigger port (Jääskeläinen et al., 2018; see Corporaal, 1997, for the book-length treatment). The one measured TTA soft-core study among the verified sources compares 32-bit cores running CHStone; the table gives both its gains and its costs (Jääskeläinen et al., 2018). Those costs belong to that 32-bit multi-unit design, not to transport triggering as such, and BrainTTA shows the model in current use for inference (Molendijk et al., 2023). VLIW gets the same parallelism by static scheduling (Fisher, 1983) but needs a costly multiport register file on FPGAs. Decoupled access/execute (DAE) splits a program into access and execute streams joined by queues; Smith's simulated gains shrink to 1.1–1.2× where the access stream needs computed values (Smith, 1984), which never happens in a dot product, and recent HLS gains come from hiding a 100-cycle memory (Metz & Själander, 2026).

This review's own synthesis, not a finding of any single source, is that static schedules suffice for regular kernels such as a dot product, while dynamic scheduling buys flexibility this kernel does not need. The results behind it: the Manchester machine was limited by its matching unit, with duplicate instructions a quarter of those executed in its integration benchmark (Gurd et al., 1985); tag matching behaves as an associative memory until restructured into directly addressed frames (Arvind & Nikhil, 1990); recent energy-minimal dataflow dropped tags for ordered tokens (Serafin et al., 2023); and TRIPS cut register-file accesses by 80–90% through direct operand forwarding, at about 11× Alpha's dynamic code size (Gebhart et al., 2009). Triggered instructions are the exception, with positive results (Parashar et al., 2013). They are set aside for schedule reasons and because no verified source shows a small FPGA implementation, not because of a measured cost, and remain an option for the processing elements of a Part 2 accelerator.

> **For LS**
>
> The plan already routes data memory straight into the MAC (LS project overview, Section 4.1), so operands bypass R0–R3 without calling the design a TTA. With one-cycle BRAM a decoupled access unit has no latency to hide, so a DAE label would only rename the design of Section 4.3. Either label earns its place only with a distinct mechanism, such as a move-based instruction set or a real access/execute queue, whose benefit is measured. Report dataflow and EDGE as considered and set aside for the costs above.

4.5 Neural-network accelerators: systolic, bit-serial, binary

Design	Principle	Reported gain	Baseline
Stripes (Judd et al., 2016)	bit-serial activations	1.92× average (ideal 16/p)	16-bit DaDianNao
Pragmatic (Albericio et al., 2017)	skip zero bits of activations	4.31×; 2.25×	16-bit; 8-bit DaDianNao
Loom (Sharify et al., 2018)	serial weights and activations	3.19×	16-bit bit-parallel
Bit Fusion (Sharma et al., 2018)	fusable 2-bit multiplier bricks	3.9× vs Eyeriss; 16× parallelism at 1–2 bits, 1× at 8 bits	Eyeriss, Stripes
UNPU (Lee et al., 2019)	lookup-table bit-serial weights	energy −27.2% at 8 bits, −53.6% at 1 bit	fixed-point MAC array
BISMO (Umuroglu et al., 2018)	AND-popcount overlay on PYNQ-Z1	6.5 binary TOPS, about 0.1 TOPS at int8 (derived)	none
Mix-GEMM (Fornt et al., 2025)	packed operands in the CPU multiplier	up to 44×; 6.7 GOP/s at 8×8 bits vs 15.8 at 2×2	unextended core
BitWave (Shi et al., 2024)	bit-column-serial with bit sparsity	up to 13.25×	sparsity-aware accelerators

Systolic and memory-centred designs. Systolic arrays are networks of nearly identical processors that compute and pass data in a fixed rhythm, so each fetched operand is used many times (Kung & Leiserson, 1978). TPU v1, the canonical int8 example, feeds 32-bit accumulators from 65,536 int8 MACs and uses systolic execution to cut on-chip buffer traffic, not to speed up the multiplier (Jouppi et al., 2017). DianNao feeds multipliers and adder trees from separate input, weight and output buffers (T. Chen et al., 2014). Eyeriss names dataflows by what stays inside a processing element and reports 0.0029 DRAM accesses per MAC on AlexNet convolutions, a precedent for a reads-per-MAC metric (Y.-H. Chen et al., 2016, 2017).

Precision-scalable designs. Most gains in the table are against 16-bit baselines and shrink at 8 bits. A benchmark of 72 precision-scalable arrays (Ibrahim et al., 2022) reports that bit-serial designs are strong when registers are shared across an array, that single bit-serial MAC units ranked worst in an earlier benchmark (cited here only through Ibrahim et al.), and that bit-serial designs have an edge at 1 GHz but not at 200 MHz. These ASIC results do not transfer to an FPGA, where the trade-off is LUT-based serial units against hard DSP48E1 slices, 240 on the XC7A100T (Xilinx, 2020). A unit serial in one 8-bit operand needs eight cycles per int8 product, so eight such units are needed to match one DSP48E1 (derived). The fair comparison is LUTs and flip-flops per sustained MAC per cycle against DSPs per MAC; Double MAC's 0.5 DSP + 11 LUT + 12 FF per MAC is one data point (Nguyen et al., 2017). BISMO's authors find bit-parallel units cheaper per bit-operation (Umuroglu et al., 2018).

Binary networks. Binarized networks replace MACs with XNOR and popcount and need 32× less weight memory, at 1.40% MNIST error against about 1.3% in float (Hubara et al., 2016). XNOR-Net's AlexNet top-1 drops from 56.6 to 44.2 when inputs are binary too, and binary weights give no speed-up on CPUs that fuse multiply and add (Rastegari et al., 2016). FINN folds batch normalization and sign into one threshold compare; its deployed SFC network (256 neurons per layer) reaches 95.83% MNIST accuracy at 12.2 k frames per second in 5,155 LUTs, while in its accuracy study a binary network with 128 neurons per layer has 6.58% error against 2.70% in float (Umuroglu et al., 2017, Table 1).

> **For LS**
>
> Operand delivery is the limit, as Kung's bound predicts, and a single bit-serial MAC is a poor choice beside a hard 8-bit multiplier at 100 MHz. Binary weights would save storage, not cycles; full binarization is a cheap optional mode whose accuracy cost must be measured.

4.6 FPGA fabric and compute-in-BRAM

XC7A100T fact	Value	Source
Slices / LUT6 / flip-flops	15,850 / 63,400 / 126,800 (four LUTs, eight FFs per slice)	Xilinx (2020)
DSP48E1 slices	240, in 3 columns of 80	Xilinx (2020, 2018)
Block RAM	135 RAMB36 (4,860 Kb), 4 columns of 40	Xilinx (2020, 2019)
DSP48E1 datapath	25-bit pre-adder, 25×18 multiplier, 48-bit accumulator; 2×24 or 4×12 SIMD only with the multiplier off	Xilinx (2018)
Block RAM ports	two independent synchronous ports; RAMB36 port widths x1–x36 in true dual-port mode, x72 in simple dual-port; each port sets its own width; optional output register adds one cycle	Xilinx (2019)
−1 grade limits	DSP fully pipelined 464.25 MHz, without MREG/ADREG 190.69 MHz; BRAM 388.20 MHz; BRAM clock-to-out 2.46 ns (0.89 ns registered)	Advanced Micro Devices (2024)

SOTA and device facts. The board's 100 MHz is far below every block limit. A one-cycle path from an unregistered BRAM read through the multiplier into the accumulator needs 2.46 + 5.48 = 7.94 ns of the 10 ns cycle before routing (derived from Advanced Micro Devices, 2024). That is plausible but tight; registering the BRAM data in the DSP's input registers leaves a wide margin. On 7-series parts the hardened direct paths are DSP-to-DSP and BRAM-to-BRAM cascades (Xilinx, 2018, 2019), so BRAM-to-DSP data uses general routing, a gap that proposed direct BRAM-DSP paths aim to close (Hu et al., 2026).

int8 packing. WP486 packs two int8 products that share one operand into a DSP48E2 for 1.75× MACs per DSP; it does not cover the DSP48E1 (Fu et al., 2017). Double MAC does it on the DSP48E1 when the shared operand is unsigned, at 0.5 DSP plus 11 LUTs and 12 flip-flops per MAC, with 14–84% network-level gains (Nguyen et al., 2017).

Compute-in-BRAM. Most of this work models changes to Intel-like fabrics; only the overlays run on commodity silicon, and a 2025 review maps the field (Arora, 2025).

Design	Idea	Key numbers	Runs on unmodified FPGAs?
CCB (Wang et al., 2021)	BRAM bitlines as bit-serial SIMD lanes	Stratix 10 peak MAC throughput 1.6–2.3× for 1.8% more die area	no (modelled)
CoMeFa (Arora et al., 2022)	operand pairs read through both ports	2.5× geomean; BRAM clock 735 ⟶ 588 MHz	no (modelled)
BRAMAC (Y. Chen & Abdelfattah, 2023)	MAC-oriented compute-in-BRAM	reports that an 8-bit MAC takes 113 cycles in CCB and CoMeFa, offset by 160 lanes per BRAM, unsigned-only	no (modelled)
M4BRAM (Y. Chen et al., 2023)	mixed-precision matrix multiply in BRAM	keeps one port free for the DSPs	no (modelled)
PiCaSO (Kabir et al., 2023)	bit-serial units beside unmodified BRAMs	540 MHz on Virtex-7; custom compute-in-BRAM fabrics "are not currently available"	yes, as an overlay
IMAGine (Kabir et al., 2024)	GEMV overlay	64K bit-serial MACs at 737 MHz, at most 0.33 TOPS at 8 bits	yes, as an overlay

For LS

The structural core is one DSP48E1 in multiply-accumulate mode fed every cycle with one input byte and one weight byte, with addresses from auto-advancing registers; the 24-bit accumulator fits in the 48-bit P register. Spec v0.1 keeps inputs and weights in two small LUT memories with registered reads (a block RAM, weights on one port and inputs on the other, is the option after Lab 12), and Vivado may fold the accumulator into the P register. A 36-bit weight port could feed four broadcast lanes (Xilinx, 2019). SIMD inside one DSP48E1 cannot multiply, so lanes means several DSPs or Double MAC packing, which suits the digits' non-negative pixel counts (derived). Compute-in-BRAM is related work only.

4.7 Quantization and requantization

State of the art. Integer-only int8 inference follows Jacob et al. (2018) and the LiteRT specification (Google AI Edge, 2026):

- real value r = S(q − Z);
- int8 weights symmetric in [−127, 127], with zero-point 0 and per-channel scales;
- int8 activations asymmetric, per tensor;
- int32 bias with scale S_x·S_w and zero-point 0.

For output neuron j with K inputs:

```
acc_j  = sum_{k=1..K} qx[k] * qw[j][k]  +  b'_j
b'_j   = b_j - Zx * sum_{k=1..K} qw[j][k]            (folded at compile time)
M_j    = Sx * Sw[j] / Sy  =  2^(-n_j) * M0_j,   M0_j in [0.5, 1)
m0_j   = round(2^B * M0_j)        B = 31 (int32) or 15 (int16); B = 17 (derived) fits the 18-bit DSP port
qy[j]  = clamp( Zy + RoundShift(acc_j * m0_j, B + n_j), -128, 127 )
RoundShift(v, s) = sign(v) * floor( (|v| + 2^(s-1)) / 2^s )   (nearest, ties away from zero)
```

Jacob et al. (2018) require round-to-nearest shifts and show that breaking ties upward ($-12/2^3$ giving -1 instead of -2) biases results and costs accuracy; an int32 M0 keeps at least 30 bits of relative accuracy. LiteRT fixes data types but not the rounding code, and gives no bit-exactness guarantee (Google AI Edge, 2026). Every rescale can be expressed as a dyadic number $b/2^c$, applied with an INT32 multiply and a shift (Yao et al., 2021; Gholami et al., 2022). Power-of-two scales reduce requantization to a rounding shift but needed up to five epochs of retraining to match float on MobileNets (Jain et al., 2020).

Accuracy. W8A8 post-training quantization costs 0.08 points on ResNet18 and 0.73 on MobileNetV2 with per-tensor scales (Nagel et al., 2021). Failures concentrate in compact models with per-layer scales: with per-layer post-training quantization MobileNet-v1 falls to 0.1% top-1, against 70.3% per-channel and 70.9% in float (Krishnamoorthi, 2018).

Accumulator sizing. The worst case is enough for LS. De Bruin et al. (2020) give $(BW_w + BW_d - 1) + \lceil \log_2 K \rceil \leq BW_{acc}$, one bit short when both operands can be −128, since $(-128) \times (-128) = 2^{14}$ needs 16 signed bits (derived); they also counted 21 instructions for a shift-round-saturate step on ARM, negligible only because each input feeds hundreds to thousands of MACs. A2Q's general bound, P ≥ α + φ(α) + 1 with α = log2 K + N + M − 1 − 1_signed, plus an $\ell 1$-norm bound, lets trained weights use narrower accumulators (Colbert et al., 2023); A2Q+ zero-centres the weights, which helps below 4 bits (Colbert et al., 2024), and AXE extends the guarantees to post-training quantization (Colbert et al., 2025). These matter only if the team narrows the accumulator.

For LS (derived from the worst-case bound)

24 bits are safe for any int8 dot product with K ≤ 511, or K ≤ 516 if weights never take −128 (largest product 16,256). A 64-input neuron needs 22 bits, leaving 2 bits for the folded bias, which the compiler must range-check because the bounds exclude it. Fan-ins of 32–64 leave few MACs to spread a software requantization over, which argues for one REQUANT instruction:

- accumulator on a DSP's 25-bit A port, m0 as a 17-bit fraction on the 18-bit B port;
- symmetric rounding with a C-port constant and the A×B rounding carry (Xilinx, 2018), whose tie behaviour must be checked against the reference model;
- shift, add Z_y and saturate in the fabric.

The P register reaches only the X and Z multiplexers, never the multiplier inputs (Xilinx, 2018, Tables 2-7 and 2-9), so REQUANT routes the accumulator through the fabric into a multiplier port: in the MAC's own DSP, at the cost of extra cycles and a cleared accumulator, or in a second DSP48E1, one more of 240 (derived). Either way it takes several cycles, about 8 with registered DSP stages and one FSM instruction (derived), counted in Section 7's end-to-end estimate. If round(2^{17}·M0) reaches 2^{17}, the compiler halves it and decrements n. Spec v0.1 starts with the simpler power-of-two scheme of CMSIS-NN (Lai et al., 2018): one four-cycle `RQ` instruction that shifts, applies ReLU and saturates, enough for the hand-made demo neuron; the LiteRT-style REQUANT above is the upgrade for trained networks.

4.8 TinyML benchmarks and datasets

Workload or budget	Size	Source
Smallest MLPerf Tiny workload (anomaly-detection autoencoder)	about 265k MACs	Tschand et al. (2025)
Keyword spotting on a microcontroller	80–500 KB memory, 6–80 million operations per second	Skillman & Edsö (2020)
Typical MCU	under 256 kB SRAM; STM32F746 has 320 KB SRAM at 216 MHz	Lin et al. (2023)
MNIST linear classifier	7,850 parameters, 12.0% error	LeCun et al. (1998)
UCI digits, 64-10 linear model	640 MACs, 650 parameters (derived)	Alpaydin & Kaynak (1998)
UCI digits, 64-32-10 MLP	2,368 MACs, 2,410 parameters (derived)	Alpaydin & Kaynak (1998)
LS data memory	4,096 addresses	LS project overview (internal document)

State of the art. The TinyML position paper describes devices that typically draw under a milliwatt with a few hundred kB of memory, and sets four initial tasks: keyword spotting, visual wake words, image classification and anomaly detection (Banbury et al., 2020). MLPerf Tiny v1.3 has five tasks with quality targets declared in advance (90%, 80%, 85% and 0.85 AUC for the original four), the fifth a streaming wake-word test that timestamps GPIO detection pulses (MLCommons, 2025). MLPerf Power timestamps a pin toggled at the start and end of inference and integrates whole-device power into energy per inference; tiny systems draw as little as 5.64 mW, and tiny efficiency rose 79× to over 1000× before flattening (Tschand et al., 2025). On the same XC7A100T −1 die, spatial dataflow runs open-division keyword spotting in 0.033 ms and 53.7 µJ using 67% of the LUTs, and anomaly detection in 0.045 ms using 86% of the DSPs (Borras et al., 2022). Ethos-U55 runs an offline-compiled command stream on 32–256 MACs per cycle (Skillman & Edsö, 2020).

Datasets. The UCI optical digits hold 5,620 8×8 images (counts 0–16) from 43 writers: 30 writers produced the 3,823 training images and a different 13 the 1,797 test images (Alpaydin & Kaynak, 1998).

> **For LS**
>
> LS sits one to two orders of magnitude below "tiny" (derived), so it cannot run MLPerf Tiny. It can adopt the method: a quality target declared in advance, and a timed window opened and closed by the program. The MNIST linear classifier does not fit in 4,096 bytes as int8, which justifies the UCI digits. Results set beside spatial FPGA designs should be normalized by resources.

4.9 Across the clusters

Theme	Convergent evidence	Contradictions and caveats	Gap
The bottleneck is operand delivery and control	energy tables (Horowitz, 2014; Y.-H. Chen et al., 2016); I/O bound (Kung, 1982); CFU ablation (Prakash et al., 2023); OPEF (Garofalo et al., 2021); Arrow's convolution result (Al Assir et al., 2021)	headline speed-ups track the baseline: 55× per operator vs 3× end to end on the Arty board (Prakash et al., 2023); 16-bit baselines in bit-serial work (Judd et al., 2016); most speed-up comes from parallelism, not overhead removal (Dally et al., 2020)	no verified measurement at 8-bit FPGA scale
Loops, post-increment and dual fetch	DSP manuals (Texas Instruments, 2001b; Analog Devices, 1995); ZOLC (Kavvadias & Nikolaidis, 2008); Octavo (LaForest et al., 2014); Arm (Yiu, 2019); Gautschi et al. (2017); Schuiki et al. (2021); Nadalini et al. (2023)	gains range from 1.07× on an FPGA (LaForest et al., 2014) to 5× analytic (Schuiki et al., 2021); eight RISC-V papers share a senior author and that group's cores and baselines (Section 4.2); the FPGA soft cores in Section 4.1 lack hardware loops, but the open RI5CY RTL has them (Gautschi et al., 2017)	no verified FPGA measurement of an open core with these mechanisms
Wide accumulator	8 guard bits in DSPs (Texas Instruments, 2001a; Analog Devices, 1995); 32-bit accumulators for 16-bit products (Jouppi et al., 2017; RISC-V International, 2025); worst-case bound (de Bruin et al., 2020) and trained bounds (Colbert et al., 2023)	the simple formula is one bit short at −128 (de Bruin et al., 2020)	bias lies outside the bounds
Precision scaling	gains concentrate at 4 bits or fewer (Albericio et al., 2017; Lee et al., 2019; Ibrahim et al., 2022)	single bit-serial units rank worst while shared-register arrays do well (Ibrahim et al., 2022); the ASIC frequency results do not transfer to FPGAs	no single-MAC FPGA CPU evidence
Static scheduling	TTA (Jääskeläinen et al., 2018); DAE (Smith, 1984)	TTA costs come from one 32-bit design; DAE's large HLS gains are latency hiding (Metz & Själander, 2026)	8-bit cases not assessed (leads in Section 9)
Benchmark method	MLPerf Tiny and Power methods (MLCommons, 2025; Tschand et al., 2025)	MLPerf Power's 20 mJ autoencoder figure conflicts with its own Fig. 7	no benchmark at 8×8-digit scale

5 Comparison matrix

Tiers: [PR] peer-reviewed, [BK] book, [VD] vendor document, [PP] preprint, [G] other gray source. “Derived” marks numbers computed here.

System	Year	Platform	Principle	MACs per cycle or speed-up (baseline)	Resources	Relevance to LS
Textbook multicycle [BK] (Harris & Harris, 2021; Patterson & Hennessy, 2021)	2021	Model	FSM, one operation per step	CPI 4.12 (SPECINT2000 mix) / 4.10 (SPECINT2006 mix); 0.5× (single-cycle)	fewer datapath units	Baseline the plan matches
KCPSM6 [VD] (Chapman, 2014)	2014	Xilinx 7-series	8-bit, fixed two-cycle instructions	CPI 2; no multiply	26 slices + 1 BRAM	8-bit reference; external same-board point
SERV [G] (Kindgren, 2018)	2018	Artix-7	Bit-serial RV32I	32 cycles per one-stage instruction	125 LUT, 164 FF (no register file)	Area floor, serial cost
FazyRV [PR] (Kissich & Baunach, 2024)	2024	7-series, iCE40	1–8-bit chunked datapath	addition 32 ⟶ 4 cycles (1 ⟶ 8 bits)	SoC in 77 slices	Width-vs-CPI knob
PicoRV32 [G] (Wolf, 2015)	2015	Kintex-7, Virtex-7	Multicycle state machine	CPI about 4; MUL 40 cycles	761–2,019 LUT	Real twin of LS FSM
VexRiscv [G] (Papon, 2017)	2017	Artix-7, fastest grade	2–5+ stage pipeline	0.52–1.21 DMIPS/MHz	504–1,418 LUT	Artix-7 area and Fmax curve
J1 [G] (Bowman, 2010)	2010	Spartan-3E	Single-cycle dual stack, one dual-port RAM for code and data	about 100 ANS Forth MIPS at 80 MHz	no LUT count	Single-cycle use of BRAM ports
RI5CY/Xpulp [PR] (Gautschi et al., 2017)	2017	65/28 nm ASIC; open RTL	Hardware loops, post-increment, SIMD dot product	+37% (loops + post-increment); 3.5× with built-ins (same core, no extensions)	+6.6 kGE	Ablation template; open core with the mechanisms
CFU Playground [PR] (Prakash et al., 2023)	2023	Arty A7-35T, 100 MHz (MobileNetV2); Fomu iCE40UP5k, 12 MHz (keyword spotting)	VexRiscv + custom MAC unit	1×1 convolutions 9.8× ⟶ 26× (< 1 cycle/MAC) ⟶ 55×; whole MobileNetV2 3×; keyword spotting 75×, 3× from the unit (same VexRiscv, no unit)	in figures only	Closest same-family ML work
Flex-RV [PR] (Ozer et al., 2024)	2024	Flexible IGZO ASIC	SERV + coupled MAC and requantize unit	no speed-up reported	12,596 NAND2 equivalents	Same instruction ideas
PULP-NN [PR] (Garofalo, Rusci, et al., 2020)	2020	PULP cluster	sdotp4 + register blocking	8.8× (plain RV32IMC); 15.5 MAC/cycle on 8 cores	none stated	MAC-to-load ratio
XpulpNN [PR] (Garofalo et al., 2021)	2021	22 nm	Load fused into MAC	OPEF 0.57 ⟶ 0.94; 1.65× (PULP-NN kernel)	core +17.5%	Fold loads into MAC

System	Year	Platform	Principle	MACs per cycle or speed-up (baseline)	Resources	Relevance to LS
CMSIS-NN [PP] (Lai et al., 2018)	2018	Cortex-M7, 216 MHz	int8 widened to int16, dual MAC	4.6× (CMSIS-DSP baseline)	none stated	Cost of no int8 MAC
P-CORE [PR] (Ali et al., 2025)	2025	Zynq UltraScale+, simulated	RISC-V packed SIMD draft	about 7× MatMul; 5.96× LeNet-5 (same core, RV32IM)	3.0× base LUTs, 170 MHz	Digit CNN on soft core
Arrow [PP] (Al Assir et al., 2021)	2021	XC7A200T-1, 100 MHz	RVV v0.9 co-processor	dot product 25–33×; convolution 1.4–1.9× (cacheless MicroBlaze; modelled)	2,715 vs 2,241 LUT	Same family and clock
TMS320C54x [VD] (Texas Instruments, 2001a, 2001b)	2001	DSP	Dual-operand MAC + repeat	1 MAC/cycle; N + 2 cycles per dot product (derived)	17×17 multiplier, 40-bit accumulator	Template
ADSP-2100 [VD] (Analog Devices, 1995)	1995	DSP	MAC with two parallel fetches + DO UNTIL	N + 6 cycles per FIR (derived)	40-bit accumulator	Template
ZOLC [PR] (Kavvadias & Nikolaidis, 2008)	2008	RISC/ASIP, 0.13 µm pre-layout	Zero-overhead loop controller	MAC loop 10 ⟶ 6 cycles; kernels −25.5% / −44% cycles	298–4,428 gates	Loop cost and benefit
Octavo [PR] (LaForest et al., 2014)	2014	Stratix IV	Address offsets + branch folding	1.07–1.92×; FIR 1.16× looping, 1.09× unrolled (MIPS-like code, same machine)	Fmax × 0.939	FPGA soft-CPU precedent
Stream semantic registers [PR] (Schuiki et al., 2021)	2021	22 nm	Address-generator streams + loops	utilization 17% ⟶ 100% (analytic)	+11% core	Utilization ladder
Flex-V [PR] (Nadalini et al., 2023)	2023	22 nm, 8 cores	MAC with load + 2-D address generator	int8 26.9 MAC/cycle (XpulpNN 26.1, RI5CY 16.6)	core +30%, cluster +6%	Stride and rollback
Snitch loop nests [PR] (Colagrande et al., 2025)	2025	RISC-V cluster	Nested zero-overhead loops	FPU utilization 85% ⟶ 96.1–99.4%	< 3% area	Outer-loop overhead
TTA soft core [PR] (Jääskeläinen et al., 2018)	2018	Zynq Z7020	Transport-triggered	single-issue 15–173% faster (MicroBlaze-5); dual-issue up to 88% faster at 67% of the resources (VLIW)	956 LUT; register file 24 LUT	Closest TTA precedent
BrainTTA [PR] (Molendijk et al., 2023)	2023	22 nm	TTA NN SoC	77 GOPS at 8 bits; 2.47 / 28.6 TOPS/W (8-bit / binary)	none stated	TTA is current
DAE [PR] (Smith, 1984)	1984	Simulation, CRAY-1 model	Decoupled access/execute	1.1–2.1×, average 1.58	none stated	Access-unit idea
DAE4HLS [PP] (Metz & Själander, 2026)	2026	AMD Vitis HLS	Explicit decoupling	decoupling alone 1.58–40.57× (Vitis)	none stated	Gains need latency
TRIPS [PR] (Gebhart et al., 2009)	2009	Custom ASIC	EDGE block dataflow	about Pentium 4 on SPEC; code ~11× Alpha	170 M transistors	Borrow forwarding, not the ISA
Triggered instructions [PR] (Parashar et al., 2013)	2013	Model	No program counter	2.0× (program-counter spatial baseline)	none stated	Uncommon control model

System	Year	Platform	Principle	MACs per cycle or speed-up (baseline)	Resources	Relevance to LS
TPU v1 [PR] (Jouppi et al., 2017)	2017	ASIC	256×256 int8 systolic	65,536 MAC/cycle; 15–30× (Haswell, K80)	28 MiB on chip	Canonical systolic design
DianNao [PR] (T. Chen et al., 2014)	2014	65 nm layout	Multipliers + adder trees, split buffers	452 GOP/s; 117.87× (SIMD CPU)	3.02 mm², 485 mW	Split operand buffers
Eyeriss [PR] (Y.-H. Chen et al., 2017)	2017	65 nm chip	Row-stationary	168 PEs; 0.0029 DRAM accesses per MAC	278 mW	Reads-per-MAC metric
Pragmatic [PR] (Albericio et al., 2017)	2017	65 nm, simulated	Zero-bit skipping	4.31× (16-bit); 2.25× (8-bit DaDianNao)	1.68× area	Gain shrinks at 8 bits
UNPU [PR] (Lee et al., 2019)	2019	65 nm chip	Lookup-table bit-serial weights	energy −27.2% at 8 bits (fixed MAC array)	16 mm² die	Weak at 8 bits
BISMO [PR] (Umuroglu et al., 2018)	2018	PYNQ-Z1 (Zynq-7020)	AND-popcount overlay	6.5 binary TOPS (about 0.1 int8 TOPS, derived)	86% LUT, 92% BRAM	FPGA bit-serial
Mix-GEMM [PR] (Fornt et al., 2025)	2025	22 nm, FPGA emulation	Operand packing in CPU multiplier	up to 44× (unextended core); 6.7 GOP/s at 8×8 bits	+2% area	Packing idea
FINN SFC [PR] (Umuroglu et al., 2017)	2017	ZC706	Binary dataflow + thresholds	12.2 k frames/s at 95.83% MNIST accuracy	5,155 LUT, 16 BRAM	Cost of a binary mode
Double MAC [PR] (Nguyen et al., 2017)	2017	Virtex-7	Two int8 MACs per DSP48E1	2 MACs per DSP per cycle; 14–84% network gain	0.5 DSP + 11 LUT + 12 FF per MAC	Packing on the LS DSP
BRAMAC [PR] (Y. Chen & Abdelfattah, 2023)	2023	Arria-10 model	Compute-in-BRAM	1.9× peak at 8 bits	+6.8% core area	Not buildable on Artix-7
PiCaSO [PR] (Kabir et al., 2023)	2023	Virtex-7, Alveo U55	Bit-serial units beside BRAM	540 MHz on Virtex-7	none stated	Buildable PIM overlay
hls4ml / FINN [PP] (Borras et al., 2022)	2022	Arty A7-100T (xc7a100t-1)	Spatial dataflow	KWS 0.033 ms, 53.7 µJ; AD 0.045 ms	67% LUT (KWS); 86% DSP (AD)	Same die, fixed function
Ethos-U55 [VD] (Skillman & Edsö, 2020)	2020	MCU NPU	Command stream + MAC engine	32–256 MAC/cycle; 50× with Cortex-M55 (Cortex-M7)	none stated	Commercial comparator

6 Closest comparators and what is uncommon at LS scale

Comparator	Shared with LS	Different from LS
CFU Playground (Prakash et al., 2023)	int8 MAC unit beside a soft CPU, board-measured, step-by-step ablation	Arty A7-35T (MNV2) and Fomu iCE40UP5k (KWS); 32-bit VexRiscv; unit reached through custom instructions; operands from DDR3 or SPI flash, not streamed from on-chip BRAM
KCPSM6 (Chapman, 2014)	8-bit soft CPU on 7-series, fixed CPI, can run on the same board	no multiply or MAC
RI5CY (Gautschi et al., 2017)	hardware loops, post-increment and dot product in one open core	32-bit; ASIC results only in the verified sources
Flex-RV (Ozer et al., 2024)	tiny CPU with MAC, bias + ReLU and requantize instructions	flexible ASIC, 4-bit model, no speed-up reported
Arrow (Al Assir et al., 2021)	Artix-7 at 100 MHz, dot products	vector co-processor on MicroBlaze, modelled cycles
Octavo (LaForest et al., 2014)	FPGA soft CPU with address and loop overhead removed in hardware	36-bit, multithreaded, Stratix IV
TTA soft core (Jääskeläinen et al., 2018)	sub-1,000-LUT soft core with a measured resource table	32-bit, CHStone benchmarks, dedicated compiler
P-CORE (Ali et al., 2025)	soft core, int8, digit recognition (LeNet-5)	32-bit packed SIMD, simulated
Borras et al. (2022)	same XC7A100T −1 die	fixed-function dataflow using most of the chip

Prior art. Every LS mechanism has been done before:

- wide-accumulator MACs with dual operand fetch (Texas Instruments, 2001b; Analog Devices, 1995);
- auto-increment addressing and hardware loops (Gautschi et al., 2017; Yiu, 2019);
- a requantize instruction (Ozer et al., 2024);
- systolic arrays (Kung & Leiserson, 1978);
- bit-serial and binary arithmetic (Kindgren, 2018; Umuroglu et al., 2017);
- DSP packing (Nguyen et al., 2017);
- the transport-triggered and decoupled models (Jääskeläinen et al., 2018; Smith, 1984).

The follow-up check found more candidate prior art outside the verified set (Section 9): an 8-bit core with added instructions, integer-only ML code compiled for 8-bit microcontrollers, 8-bit microcontrollers with MAC hardware, and an 8-bit-capable transport-triggered family. Until those are checked, the report should claim none of these as new.

Which cores were examined. The FPGA soft cores in the Section 4.1 table document no hardware loops or address generators, but that follows from which cores were chosen: the open RI5CY RTL has two hardware-loop levels and post-increment loads (Gautschi et al., 2017), Octavo added such mechanisms to an FPGA soft processor (LaForest et al., 2014), and CFU Playground moved the accumulation loop into a custom unit (Prakash et al., 2023). Missing is a verified measurement of an open core with these mechanisms on the LS board (lead in Section 9). Three of the four projects that inspired LS (Tiny TPU, TinyTinyTPU and tiny-tpu) already build 2×2 to 8×8 systolic arrays, as listed in the LS project overview (Section 8), so a small array alone would not stand out.

Uncommon within the verified sources. No 8-bit soft CPU on an FPGA was found that streams int8 operands from dual-port BRAM into a hard DSP MAC under a hardware loop, and no peer-reviewed int8 MLP result on the UCI digits was found. Absence from this search is not proof of absence.

What would be original for a student project. The combination and its measurement: an 8-bit soft core on Artix-7 that streams int8 operands from two operand memories into a DSP48E1 MAC under a hardware repeat, with an int8 requantize, measured on one bitstream against the best conventional code for the same core (optionally also against KCPSM6), and bit-exact against an integer reference model. This reproduces known mechanisms at a smaller scale, a legitimate contribution rather than a new architectural idea. Input-broadcast lanes for the digit network, Double MAC packing and an in-ISA binary mode with threshold activation would be measured extras.

7 Implications for the LS design decisions

Structural score. Each candidate, as it would be built for LS, against the criterion of Section 3.5:

Candidate	S1: no fetch or decode per MAC	S2: operands every cycle	S3: sum stays in MAC unit	Note
1. Multi-cycle register machine + MAC (rungs L and U)	no	no	yes	4 cycles per MAC unrolled, 8 in a loop; the multiplier idles at least 3 cycles in 4 (derived)
2. Hardware repeat + address generators + two operand memories (rung S, `MAC #n`)	yes	yes	yes	one MAC per cycle after setup
3. Transport-triggered	no	yes, with two load units	yes	one move instruction per cycle even when software-pipelined; adding a hardware loop turns it into option 2
4. Decoupled access/execute	no	yes	yes	the execute stream still issues one MAC per element unless looped in hardware
5. Single-cycle dual-stack, same memory and MAC as option 2	no	yes	yes	J1 as published fails S2 and S3: one shared RAM, no multiplier (Bowman, 2010)
6. Input-broadcast MAC lanes (L = 2–4) under a hardware repeat	yes	yes, with an x18 or x36 weight port or one BRAM per lane	yes	packed SIMD in 8-bit registers would fail S2 and S3
7. Bit-serial MAC	depends on control	yes (bits per cycle), but only 1/8 int8 MAC per cycle per unit	yes	eight serial units are needed to match one DSP (derived)
8. Tiny CPU + small systolic array	yes	only with one weight byte per processing element per cycle	yes if output-stationary	at batch 1 it behaves like broadcast lanes (Section 3.2)

1. Multi-cycle register machine + MAC (current plan)	
Buys	Smallest control, easiest verification; fewer datapath components (Patterson & Hennessy, 2021; Harris & Harris, 2021)
Risks at LS scale	CPI 4; 4 cycles per MAC unrolled, 8 in a loop, 20 without a MAC instruction (derived; Section 3.1); fails S1
Fair measurement	Rungs B, L and U, same multiplier
Verdict	**Keep for control and as the baseline**
2. DSP-style zero-overhead loops + address generators + dual-port BRAM streaming	
Buys	N + 2 cycles per dot product on the C54x (derived from Texas Instruments, 2001b); +37% average, up to +41%, from loops and post-increment on one core (Gautschi et al., 2017); 1.07–1.92× on an FPGA soft CPU, but only 1.16× looping and 1.09× unrolled on its FIR (LaForest et al., 2014, Table I)
Risks at LS scale	1–2 BRAM read-latency cycles (Xilinx, 2019); one-cycle BRAM-to-MAC path 7.94 of 10 ns (derived); Fmax cost (LaForest et al., 2014); DSP-style hardware is "difficult for compilers to use effectively" (Eyre & Bier, 2000), mitigated because the LS compiler only emits a few fixed dot-product templates (setup, repeat-MAC, requantize, store); step mode (resolved below)
Fair measurement	Rung S against rungs U, L and B on one bitstream (optionally KCPSM6); cycles per MAC, utilization, post-route resources, WNS and Vivado power with the repeat on and off
Verdict	**Recommended core for Part 1 (adopted as `MAC #n` in spec v0.1)**
3. Transport-triggered	
Buys	In one 32-bit study: register-file savings, 15–173% faster than MicroBlaze-5, and a dual-issue TTA at 67% of a VLIW's resources running up to 88% faster (Jääskeläinen et al., 2018); in current use for NN inference (Molendijk et al., 2023)
Risks at LS scale	Costs reported for that design: 43-bit instructions, a 265-LUT interconnect, programs 21–49% larger than VLIW, a dedicated compiler (Jääskeläinen et al., 2018); fails S1 without a hardware loop
Fair measurement	Same function units against an operation-triggered baseline, resource table in Jääskeläinen et al.'s format
Verdict	**Not a separate design for LS: the plan's data-memory-to-MAC path already moves operands straight into the MAC**
4. Decoupled access/execute	
Buys	Dot-product addresses never depend on data, so decoupling is complete (Smith, 1984)
Risks at LS scale	Little latency to hide with 1-cycle BRAM; large HLS gains come from a 100-cycle memory (Metz & Själander, 2026); deadlock risk if the streams interact (Smith, 1984)
Fair measurement	Cycles per inner-loop iteration, baseline vs decoupled, as in Smith's Table I
Verdict	**Same hardware as option 2 at this latency; name it only if a real queue is built and its benefit measured**
5. Single-cycle dual-stack machine (J1-like)	
Buys	CPI about 1 on BRAM, dense code, tiny decode (Bowman, 2010; Koopman, 1989)
Risks at LS scale	J1's missing multiplier and shared code/data RAM are costs of that implementation (Bowman, 2010); with the same Harvard memory and MAC as option 2 the inner loop is the same streaming MAC, so the model changes code density and decode, not MAC throughput (derived); non-refereed source, no LUT counts
Fair measurement	Cycles per MAC and code size against option 2 with the same memory and MAC
Verdict	**Good control core; no inner-loop advantage**

6. SIMD lanes, built as input-broadcast MAC lanes (L = 2–4)	
Buys	Packed SIMD is standard on 32-bit cores (RISC-V International, 2025, 2026a), with 4.8× (fully connected) to about 7× (MatMul) in P-CORE (Ali et al., 2025); most speed-up comes from parallelism (Dally et al., 2020); each input feeds every neuron, so L lanes need L weight bytes plus one input byte per cycle (Section 3.2)
Risks at LS scale	DSP48E1 SIMD cannot multiply (Xilinx, 2018), so each lane is a DSP, or half of one with Double MAC if the shared operand is unsigned (Nguyen et al., 2017); L accumulators and a REQUANT per lane; the compiler must interleave weights; no gain for the single demo neuron
Fair measurement	Digit network end to end, roughly 1,300–1,500 cycles for L = 4 against about 3,800 for L = 1 (derived, below); DSPs, LUTs and BRAMs per sustained MAC
Verdict	**Measured Part 2 step on top of option 2**
7. Bit-serial precision-scalable MAC (incl. binary XNOR-popcount)	
Buys	Small units (Kindgren, 2018); gains at 4 bits or fewer (Lee et al., 2019; Sharma et al., 2018); 8 binary MACs per byte-wide XNOR-popcount (derived); threshold activation (Umuroglu et al., 2017)
Risks at LS scale	ASIC findings on single bit-serial units and clock frequency (as reported by Ibrahim et al., 2022) do not transfer to an FPGA; int8 needs eight serial units per DSP48E1 equivalent (derived); 2.25× at 8 bits (Albericio et al., 2017); data-dependent cycle counts; small binary networks lose accuracy (Umuroglu et al., 2017)
Fair measurement	LUTs and FFs per sustained MAC against DSPs per MAC; accuracy on the UCI test set
Verdict	**At most an optional binary mode**
8. Tiny CPU + small systolic array	
Buys	Operand reuse beyond 1 MAC per cycle (Kung, 1982; Jouppi et al., 2017); DSP cascades, 16 of 240 DSPs for 4×4 (Xilinx, 2018)
Risks at LS scale	At batch 1 each weight is used once, so 16 processing elements need 16 weight bytes per cycle: two RAMB36 at 72 bits or four at 36 bits (derived from Xilinx, 2019); so fed, a 4×4 array acts like 16 broadcast lanes, and its second dimension pays off only with batch or convolution reuse (derived); common in student projects (LS project overview, Section 8); schedule risk
Fair measurement	Sustained vs peak MACs per cycle and reads per MAC (Y.-H. Chen et al., 2017), against option 6 at equal BRAM bandwidth
Verdict	**Part 2 extension**

The one measured claim. On the Nexys A7-100T at 100 MHz, report cycles per int8 MAC and total cycles for a dot product, read from an on-chip cycle counter that the program starts and stops; utilization is MACs divided by cycles (Schuiki et al., 2021). On a deterministic core these counts are fixed by the RTL, so state them before measuring, confirm them in simulation, and use the board for what simulation cannot give: post-route resources, timing and Vivado's power estimate per mechanism. The target claim: `MAC #n` sustains one cycle per int8 MAC, against 4 for the best conventional code (an unrolled classic MAC), 8 for a MAC loop and 20 without a MAC instruction; for the 64-input neuron, 112 cycles against 300 (2.68×, predicted), at a stated cost in LUTs, flip-flops, DSPs and slack.

Step	What to do	Prediction or rule
Predictions first	Publish the rung table (Section 3.1) before any run; this follows Gautschi et al.'s (2017) same-core method and avoids model-only cycle counts (Al Assir et al., 2021; Ali et al., 2025)	B 1,328, L 560, U 300 and S 112 cycles for 64 inputs (derived from the spec v0.1 listings)
Optional external point on the same board	KCPSM6 (Chapman, 2014) driving the same DSP48E1 MAC as an I/O peripheral; KCPSM6 is Xilinx copyright material whose user guide carries a proprietary notice, so record its source and the licence terms supplied with the download in the attribution section, as the course brief requires for code the team did not write	about 16 cycles per MAC: eight two-cycle instructions (two operand reads, two port writes, two pointer updates, a decrement, a branch; derived)
One bitstream, two builds	Run all four programs on one bitstream, so only the code differs; build a second bitstream with the repeat disabled in the decoder; report LUT, FF, DSP and BRAM use, worst negative slack at speed grade −1, and Vivado's report_power for both	power is a Vivado estimate with its activity source stated (vectorless defaults, or testbench switching activity), and so is any energy per MAC derived from it; "measured" is kept for meter readings; the course brief requires the figures Vivado actually gave
Two lengths	N = 32 and 64 (optionally 4 and 16), with inputs preloaded in memory, not read from the switches	the slope gives the cost per input with the fixed cost removed
End-to-end per UCI digit	64-32-10 network including requantization and argmax: about $32 \times (64 + 4 + o) + 10 \times (32 + 4 + o) + A$ cycles, with $o \approx 28$ for per-neuron set-up, requantization and store (about seven instructions) and $A \approx 120$ for the argmax	`MAC #n` about 3,800 cycles; the MAC loop about 20,400; four input-broadcast lanes roughly 1,300–1,500, about 2.5–3× rather than 4×, because set-up, requantization, argmax and the 10-neuron output layer (10 of 12 lane slots used) stay partly serial (all derived)
Secondary result	Bit-exact agreement with an integer reference model of the int8 pipeline on the 1,797 UCI test images, with accuracy against float	quality target declared beforehand

The gap between per-kernel and end-to-end gains is the pattern CFU Playground reports (Prakash et al., 2023).

Step mode. Make the repeat count part of the control FSM: the repeat instruction enters a streaming state that loops on itself while the count is non-zero, and in step mode each button press advances that state by one MAC. One press is then one MAC iteration, the repeat appears as states in the FSM diagram, and the "finite state machine at its core" that the course brief requires stays visible.

8 Must-cite shortlist for the report

Reference	Where to cite it in the report
Hennessy & Patterson (2019)	Introduction: why domain-specific hardware and low-precision integers
Dally et al. (2020)	Design rationale: instruction overhead, parallelism as the source of speed-up, the FPGA efficiency penalty
Horowitz (2014)	Motivation: energy of an int8 MAC against instruction fetch and memory access
Williams et al. (2009)	Analysis: the bandwidth bound behind one MAC per cycle and the lane option
Harris & Harris (2021)	Baseline: iron-law arithmetic and multicycle CPI
Eyre & Bier (2000)	Related work: the DSP recipe for one MAC per cycle, and its cost for compilers
Texas Instruments (2001b)	Design: the dual-operand MAC with repeat as the cycle budget to match
Gautschi et al. (2017)	Method: hardware loops and post-increment measured on one core
LaForest et al. (2014)	Related work: overhead removal in an FPGA soft CPU, its small FIR gain and its Fmax cost
Chapman (2014)	Evaluation (optional): KCPSM6 as the 8-bit external same-board point
Prakash et al. (2023)	Related work and threats to validity: closest FPGA ML soft-CPU work and its baseline effects
Jacob et al. (2018)	Quantization: the integer-only requantization formula and rounding rule
de Bruin et al. (2020)	Datapath sizing: the worst-case accumulator bound, with the −128 correction, behind the 24 bits
Xilinx (2018)	Implementation: DSP48E1 multiplier, accumulator paths and rounding
Xilinx (2019)	Implementation: dual-port BRAM, port widths and read latency

9 Limitations of this review

- **Coverage.** Google Scholar was not queried and several databases were blocked (Section 2), so some 2023–2026 FPGA soft-CPU papers may be missing. The main search missed 8-bit microcontrollers with MAC hardware, ISA extensions to 8-bit cores and compilers for KB-sized models; the follow-up check covered them, hence the narrower claims in Sections 1 and 6.
- **Leads not verified within this review's reference set.** None supports any claim above; check each before the report cites it.

Lead	What it appears to be	Where to start
8-bit AVR-based RISC processor for elliptic-curve cryptography (MICRO workshops, 2012)	ISA extension on an 8-bit datapath	https://doi.org/10.1109/MICROW.2012.20
SeeDot (PLDI 2019)	compiler from trained KB-sized models to fixed-point code for microcontrollers, including the Arduino Uno, and low-end FPGAs	https://doi.org/10.1145/3314221.3314597
PSoC 1 (Cypress/Infineon); C8051F12x (Silicon Labs)	8-bit microcontrollers reported to include hardware MAC blocks	the vendors' official datasheets

Lead	What it appears to be	Where to start
MAXQ family	described by its vendor as transport-triggered, with 16-bit instructions and 8-bit parts	https://wcm-cce.cldnet.analog.com/media/en/technical-documentation/user-guides/maxq-family-users-guide.pdf
CV32E40P in the CORE-V-MCU	open RI5CY descendant; the documentation lists the Nexys A7 (XC7A100T) as a target; check in its manual and RTL whether the PULP extensions are enabled and verified	https://core-v-mcu-documentation.readthedocs.io/en/stable/doc-src/overview.html
Precision-scalable MAC benchmark (IEEE JETCAS, 2019)	the earlier benchmark cited here only through Ibrahim et al.	https://doi.org/10.1109/JETCAS.2019.2950386
arXiv:2511.21232 (2025)	RISC-V custom unit for depthwise separable convolution on an Artix-7	arXiv
MLPerf Tiny, NeurIPS 2021 Datasets and Benchmarks	proceedings version, whose printed author list differs from the arXiv version	https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html

- **Versions and depth.** Some numbers come from arXiv or author versions (for example TPU, XpulpNN, PULP-NN, Flex-V, FINN, BISMO, Mix-GEMM and XNOR-Net), and Eyre and Bier (2000), a magazine overview, was read in its BDTI white-paper version. CCB, BrainTTA, triggered instructions, BitWave and the 2025 compute-in-memory review were read at abstract level, the Corporaal and Lapsley books only at catalogue level. Section 4.5 of Patterson and Hennessy (2021) is an online section read on the publisher's companion site; the Harris and Harris (2021) figures were checked against the authors' Chapter 7 companion slides.
- **Transferability.** Most evidence is ASIC or simulation at 12–65 nm, and FPGA logic is 10–100× less efficient (Dally et al., 2020).
- **Source concentration.** Eight of the ISA-extension and DSP-mechanism papers share a senior author (Section 4.2).
- **Dropped works.** The TensorFlow Lite Micro paper, the NEORV32 datasheet, the DSP56000 manual and the MLPerf Tiny paper failed metadata checks; they could be repaired and re-checked.
- **Source inconsistencies.** MLPerf Power's 20 mJ figure conflicts with its own plot. CoMeFa states two clock overheads for one variant. Later papers restate CCB's area cost four ways; CCB's own 1.8% is used here. Nagel et al.'s "within 0.7%" text conflicts with their Table 6.
- **Derived numbers.** The rung table, the KCPSM6, end-to-end and lane predictions, the 7.94 ns path, the 0.3 pJ MAC, the accumulator limits and the REQUANT cost are calculations, not measurements. LiteRT's rounding code was not checked, so bit-exactness must be defined against the team's own reference model.
- **Plan snapshot.** LS details come from the LS project overview as of 8 November 2026. The LS design document (spec v0.1, 9 November 2026) adopts the hardware repeat as `MAC #n` and fixes the instruction set; Sections 1, 3.1, 6 and 7 use its cycle counts, which supersede the overview-based estimates.

References

Advanced Micro Devices, Inc. (2024). *Artix 7 FPGAs data sheet: DC and AC switching characteristics* (DS181, v1.27.1, July 3, 2024). https://docs.amd.com/v/u/en-US/ds181_Artix_7_Data_Sheet

Al Assir, I., El Iskandarani, M., Al Sandid, H. R., & Saghir, M. A. R. (2021). *Arrow: A RISC-V vector accelerator for machine learning inference* (arXiv:2107.07169) [Preprint; presented at the Fifth Workshop on Computer Architecture Research with RISC-V (CARRV 2021), co-located with ISCA 2021]. arXiv. https://doi.org/10.48550/arXiv.2107.07169

Albericio, J., Delmás, A., Judd, P., Sharify, S., O'Leary, G., Genov, R., & Moshovos, A. (2017). Bit-pragmatic deep neural network computing. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-50)* (pp. 382-394). Association for Computing Machinery. https://doi.org/10.1145/3123939.3123982

Ali, M., Aliagha, E., Elnashar, M., & Göhringer, D. (2025). P-CORE: Exploring RISC-V packed-SIMD extension for CNNs. *IEEE Access, 13*, 146603–146616. https://doi.org/10.1109/ACCESS.2025.3600360

Alpaydin, E., & Kaynak, C. (1998). *Optical recognition of handwritten digits* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C50P49

Analog Devices. (1995). *ADSP-2100 family user's manual* (3rd ed.). Analog Devices, Inc.

Arora, A. (2025). Compute-in-memory on FPGAs for deep learning: A review. In *2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (pp. 243-253). IEEE. https://doi.org/10.1109/FCCM62733.2025.00012

Arora, A., Anand, T., Borda, A., Sehgal, R., Hanindhito, B., Kulkarni, J., & John, L. K. (2022). CoMeFa: Compute-in-memory blocks for FPGAs. In *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (pp. 1-9). IEEE. https://doi.org/10.1109/FCCM53951.2022.9786179

Arvind, & Nikhil, R. S. (1990). Executing a program on the MIT tagged-token dataflow architecture. *IEEE Transactions on Computers, 39*(3), 300–318. https://doi.org/10.1109/12.48862

Backus, J. (1978). Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Communications of the ACM, 21*(8), 613-641. https://doi.org/10.1145/359576.359579

Banbury, C. R., Reddi, V. J., Lam, M., Fu, W., Fazel, A., Holleman, J., Huang, X., Hurtado, R., Kanter, D., Lokhmotov, A., Patterson, D., Pau, D., Seo, J.-s., Sieracki, J., Thakker, U., Verhelst, M., & Yadav, P. (2020). *Benchmarking TinyML systems: Challenges and direction* (arXiv:2003.04821v4) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2003.04821

Borras, H., Di Guglielmo, G., Duarte, J., Ghielmetti, N., Hawks, B., Hauck, S., Hsu, S.-C., Kastner, R., Liang, J., Meza, A., Muhizi, J., Nguyen, T., Roy, R., Tran, N., Umuroglu, Y., Weng, O., Yokuda, A., & Blott, M. (2022). *Open-source FPGA-ML codesign for the MLPerf Tiny benchmark* (arXiv:2206.11791) [Preprint; contribution to the 3rd Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware (MLBench) at MLSys 2022]. arXiv. https://doi.org/10.48550/arXiv.2206.11791

Bowman, J. (2010). *J1: A small Forth CPU core for FPGAs* [Non-refereed paper]. EuroForth 2010. https://www.complang.tuwien.ac.at/anton/euroforth/ef10/papers/bowman.pdf

Chapman, K. (2014). *PicoBlaze for Spartan-6, Virtex-6, 7-Series, Zynq and UltraScale devices (KCPSM6): Including ultra-compact UART macros and reference designs* (Release 9) [User guide]. Xilinx. https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/picoblaze.html

Chen, T., Du, Z., Sun, N., Wang, J., Wu, C., Chen, Y., & Temam, O. (2014). DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '14)* (pp. 269-284). Association for Computing Machinery. https://doi.org/10.1145/2541940.2541967

Chen, Y., & Abdelfattah, M. S. (2023). BRAMAC: Compute-in-BRAM architectures for multiply-accumulate on FPGAs. In *2023 IEEE 31st Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (pp. 52-62). IEEE. https://doi.org/10.1109/FCCM57271.2023.00015

Chen, Y., Dotzel, J., & Abdelfattah, M. S. (2023). M4BRAM: Mixed-precision matrix-matrix multiplication in FPGA block RAMs. In *2023 International Conference on Field Programmable Technology (ICFPT)* (pp. 69-78). IEEE.

https://doi.org/10.1109/ICFPT59805.2023.00013

Chen, Y.-H., Emer, J., & Sze, V. (2016). Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)* (pp. 367-379). IEEE. https://doi.org/10.1109/ISCA.2016.40

Chen, Y.-H., Krishna, T., Emer, J. S., & Sze, V. (2017). Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits, 52*(1), 127-138. https://doi.org/10.1109/JSSC.2016.2616357

Colagrande, L., Leone, L., Coco, M., Deaconeasa, A., & Benini, L. (2025). Towards zero-stall matrix multiplication on energy-efficient RISC-V clusters for machine learning acceleration. In *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)* (pp. 1-7). IEEE. https://doi.org/10.1109/ISLPED65674.2025.11261759

Colbert, I., Franco, G., Grob, F., Zhang, J., & Saab, R. (2025). Accumulator-aware post-training quantization for large language models. *Transactions on Machine Learning Research.* https://openreview.net/forum?id=p6l0579yj7

Colbert, I., Pappalardo, A., & Petri-Koenig, J. (2023). A2Q: Accumulator-aware quantization with guaranteed overflow avoidance. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (pp. 16943-16952). IEEE. https://doi.org/10.1109/ICCV51070.2023.01558

Colbert, I., Pappalardo, A., Petri-Koenig, J., & Umuroglu, Y. (2024). A2Q+: Improving accumulator-aware weight quantization. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, & F. Berkenkamp (Eds.), *Proceedings of the 41st International Conference on Machine Learning* (Vol. 235, pp. 9275-9291). PMLR. https://proceedings.mlr.press/v235/colbert24a.html

Corporaal, H. (1997). *Microprocessor architectures: From VLIW to TTA*. John Wiley & Sons.

Dally, W. J., Turakhia, Y., & Han, S. (2020). Domain-specific hardware accelerators. *Communications of the ACM, 63*(7), 48-57. https://doi.org/10.1145/3361682

de Bruin, B., Zivkovic, Z., & Corporaal, H. (2020). Quantization of deep neural networks for accumulator-constrained processors. *Microprocessors and Microsystems, 72*, Article 102872. https://doi.org/10.1016/j.micpro.2019.102872

Dupuis, T., Fournier, Y., AskariHemmat, M., El Zarif, N., Leduc-Primeau, F., David, J. P., & Savaria, Y. (2023). Sparq: A custom RISC-V vector processor for efficient sub-byte quantized inference. In *2023 21st IEEE Interregional NEWCAS Conference (NEWCAS)* (pp. 1–5). IEEE. https://doi.org/10.1109/NEWCAS57931.2023.10198172

Eyre, J., & Bier, J. (2000). The evolution of DSP processors. *IEEE Signal Processing Magazine, 17*(2), 43-51. https://doi.org/10.1109/79.826411

Fisher, J. A. (1983). Very long instruction word architectures and the ELI-512. In *Proceedings of the 10th Annual International Symposium on Computer Architecture (ISCA '83)* (pp. 140–150). ACM. https://doi.org/10.1145/800046.801649

Fornt, J., Reggiani, E., Fontova-Musté, P., Rodas, N., Pappalardo, A., Unsal, O. S., Cristal Kestelman, A., Altet, J., Moll, F., & Abella, J. (2025). Mix-GEMM: Extending RISC-V CPUs for energy-efficient mixed-precision DNN inference using binary segmentation. *IEEE Transactions on Computers, 74*(2), 582-596. https://doi.org/10.1109/TC.2024.3500369

Fu, Y., Wu, E., Sirasao, A., Attia, S., Khan, K., & Wittig, R. (2017). *Deep learning with INT8 optimization on Xilinx devices* (White Paper WP486, v1.0.1, April 24, 2017). Xilinx, Inc. https://docs.amd.com/v/u/en-US/wp486-deep-learning-int8

Garofalo, A., Rusci, M., Conti, F., Rossi, D., & Benini, L. (2020). PULP-NN: Accelerating quantized neural networks on parallel ultra-low-power RISC-V processors. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences, 378*(2164), Article 20190155. https://doi.org/10.1098/rsta.2019.0155

Garofalo, A., Tagliavini, G., Conti, F., Benini, L., & Rossi, D. (2021). XpulpNN: Enabling energy efficient and flexible inference of quantized neural networks on RISC-V based IoT end nodes. *IEEE Transactions on Emerging Topics in Computing, 9*(3), 1489–1505. https://doi.org/10.1109/TETC.2021.3072337

Garofalo, A., Tagliavini, G., Conti, F., Rossi, D., & Benini, L. (2020). XpulpNN: Accelerating quantized neural networks on RISC-V processors through ISA extensions. In *2020 Design, Automation & Test in Europe Conference*

& Exhibition (DATE) (pp. 186–191). IEEE. https://doi.org/10.23919/DATE48585.2020.9116529

Gautschi, M., Schiavone, P. D., Traber, A., Loi, I., Pullini, A., Rossi, D., Flamand, E., Gurkaynak, F. K., & Benini, L. (2017). Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 25*(10), 2700–2713. https://doi.org/10.1109/TVLSI.2017.2654506

Gebhart, M., Maher, B. A., Coons, K. E., Diamond, J., Gratz, P., Marino, M., Ranganathan, N., Robatmili, B., Smith, A., Burrill, J., Keckler, S. W., Burger, D., & McKinley, K. S. (2009). An evaluation of the TRIPS computer system. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XIV)* (pp. 1–12). ACM. https://doi.org/10.1145/1508244.1508246

Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M. W., & Keutzer, K. (2022). A survey of quantization methods for efficient neural network inference. In G. K. Thiruvathukal, Y.-H. Lu, J. Kim, Y. Chen, & B. Chen (Eds.), *Low-power computer vision: Improve the efficiency of artificial intelligence* (pp. 291-326). Chapman and Hall/CRC. https://doi.org/10.1201/9781003162810-13

Google AI Edge. (2026, May 28). *LiteRT 8-bit quantization specification*. Google for Developers. Retrieved November 9, 2026, from https://developers.google.com/edge/litert/conversion/tensorflow/quantization/quantization_spec

Gray, J. (2016). *GRVI Phalanx: A massively parallel RISC-V FPGA accelerator accelerator* [Extended abstract]. 2nd International Workshop on Overlay Architectures for FPGAs (OLAF 2016), Monterey, CA, United States. arXiv. https://arxiv.org/abs/1606.01037

Gray, J., & Smith, A. (2018). *Towards an area-efficient implementation of a high ILP EDGE soft processor* (arXiv:1803.06617) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.1803.06617

Gurd, J. R., Kirkham, C. C., & Watson, I. (1985). The Manchester prototype dataflow computer. *Communications of the ACM, 28*(1), 34–52. https://doi.org/10.1145/2465.2468

Harris, S. L., & Harris, D. (2021). *Digital design and computer architecture: RISC-V edition*. Morgan Kaufmann.

Heinz, C., Lavan, Y., Hofmann, J., & Koch, A. (2019). A catalog and in-hardware evaluation of open-source drop-in compatible RISC-V softcore processors. In *2019 International Conference on ReConFigurable Computing and FPGAs (ReConFig)* (pp. 1-8). IEEE. https://doi.org/10.1109/ReConFig48160.2019.8994796

Hennessy, J. L., & Patterson, D. A. (2019). A new golden age for computer architecture. *Communications of the ACM, 62*(2), 48-60. https://doi.org/10.1145/3282307

Horowitz, M. (2014). 1.1 Computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)* (pp. 10-14). IEEE. https://doi.org/10.1109/ISSCC.2014.6757323

Hu, J., Sunketa, R. R., Boutros, A., & Arora, A. (2026). *Boosting FPGA performance with direct BRAM-DSP paths* (arXiv:2607.05756) [Preprint]. arXiv. https://arxiv.org/abs/2607.05756

Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., & Bengio, Y. (2016). Binarized neural networks. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems* (Vol. 29). Curran Associates. https://proceedings.neurips.cc/paper/2016/hash/d8330f857a17c53d217014ee776bfd50-Abstract.html

Ibrahim, E. M., Mei, L., & Verhelst, M. (2022). Taxonomy and benchmarking of precision-scalable MAC arrays under enhanced DNN dataflow representation. *IEEE Transactions on Circuits and Systems I: Regular Papers, 69*(5), 2013-2024. https://doi.org/10.1109/TCSI.2022.3141519

Jääskeläinen, P., Tervo, A., Payá Vayá, G., Viitanen, T., Behmann, N., Takala, J., & Blume, H. (2018). Transport-triggered soft cores. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (pp. 83–90). IEEE. https://doi.org/10.1109/IPDPSW.2018.00022

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2704-2713). IEEE. https://doi.org/10.1109/CVPR.2018.00286

Jain, S. R., Gural, A., Wu, M., & Dick, C. H. (2020). Trained quantization thresholds for accurate and efficient fixed-point inference of deep neural networks. *Proceedings of Machine Learning and Systems, 2*, 112-128.

https://proceedings.mlsys.org/paper_files/paper/2020/hash/1eb34d662b67a14e3511d0dfd78669be-Abstract.html

Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., Boyle, R., Cantin, P.-L., Chao, C., Clark, C., Coriell, J., Daley, M., Dau, M., Dean, J., Gelb, B., ... Yoon, D. H. (2017). In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)* (pp. 1-12). Association for Computing Machinery. https://doi.org/10.1145/3079856.3080246

Judd, P., Albericio, J., Hetherington, T., Aamodt, T. M., & Moshovos, A. (2016). Stripes: Bit-serial deep neural network computing. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* (pp. 1-12). IEEE. https://doi.org/10.1109/MICRO.2016.7783722

Kabir, M. A., Kabir, E., Hollis, J., Levy-Mackay, E., Panahi, A., Bakos, J., Huang, M., & Andrews, D. (2023). FPGA processor in memory architectures (PIMs): Overlay or overhaul? In *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)* (pp. 109-115). IEEE. https://doi.org/10.1109/FPL60245.2023.00023

Kabir, M. A., Kamucheka, T., Fredricks, N., Mandebi, J., Bakos, J., Huang, M., & Andrews, D. (2024). IMAGine: An in-memory accelerated GEMV engine overlay. In *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)* (pp. 220-226). IEEE. https://doi.org/10.1109/FPL64840.2024.00038

Kavvadias, N., & Nikolaidis, S. (2008). Elimination of overhead operations in complex loop structures for embedded microprocessors. *IEEE Transactions on Computers, 57*(2), 200-214. https://doi.org/10.1109/TC.2007.70790

Kindgren, O. (2018). *SERV: The SErial RISC-V CPU* [Computer software and user manual]. GitHub. Retrieved November 9, 2026, from https://github.com/olofk/serv

Kissich, M., & Baunach, M. (2024). FazyRV: Closing the gap between 32-bit and bit-serial RISC-V cores with a scalable implementation. In *Proceedings of the 21st ACM International Conference on Computing Frontiers (CF '24)* (pp. 240-248). ACM. https://doi.org/10.1145/3649153.3649195

Koopman, P. J., Jr. (1989). *Stack computers: The new wave*. Ellis Horwood. https://users.ece.cmu.edu/~koopman/stack_computers/

Krishnamoorthi, R. (2018). *Quantizing deep convolutional networks for efficient inference: A whitepaper* (arXiv:1806.08342). arXiv. https://doi.org/10.48550/arXiv.1806.08342

Kung, H. T. (1982). Why systolic architectures? *Computer, 15*(1), 37-46. https://doi.org/10.1109/MC.1982.1653825

Kung, H. T., & Leiserson, C. E. (1978). *Systolic arrays for (VLSI)* (Report No. CMU-CS-79-103). Department of Computer Science, Carnegie-Mellon University. https://www.eecs.harvard.edu/htk/static/files/1978-cmu-cs-report-kung-leiserson.pdf

LaForest, C. E., Anderson, J., & Steffan, J. G. (2014). Approaching overhead-free execution on FPGA soft-processors. In *2014 International Conference on Field-Programmable Technology (FPT)* (pp. 99-106). IEEE. https://doi.org/10.1109/FPT.2014.7082760

Lai, L., Suda, N., & Chandra, V. (2018). *CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs* (arXiv:1801.06601). arXiv. https://doi.org/10.48550/arXiv.1801.06601

Lapsley, P., Bier, J., Shoham, A., & Lee, E. A. (1997). *DSP processor fundamentals: Architectures and features*. IEEE Press. https://doi.org/10.1109/9780470544433

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278-2324. https://doi.org/10.1109/5.726791

Lee, J., Kim, C., Kang, S., Shin, D., Kim, S., & Yoo, H.-J. (2019). UNPU: An energy-efficient deep neural network accelerator with fully variable weight bit precision. *IEEE Journal of Solid-State Circuits, 54*(1), 173-185. https://doi.org/10.1109/JSSC.2018.2865489

Levy, B., & Koch, M. (2020). *FemtoRV32/FemtoSOC: A minimalistic RISC-V CPU* [Computer software; part of learn-fpga]. GitHub. Retrieved November 9, 2026, from https://github.com/BrunoLevy/learn-fpga/tree/master/FemtoRV

Lin, J., Zhu, L., Chen, W.-M., Wang, W.-C., & Han, S. (2023). Tiny machine learning: Progress and futures. *IEEE*

Circuits and Systems Magazine, 23(3), 8-34. https://doi.org/10.1109/MCAS.2023.3302182

Metz, D., & Själander, M. (2026). *DAE4HLS: Exposing memory-level parallelism for high-level synthesis using explicit decoupling* (arXiv:2605.23549) [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2605.23549

MLCommons. (2025, September 24). *A new TinyML streaming benchmark for MLPerf Tiny v1.3.* https://mlcommons.org/2025/09/mlperf-tiny-v1-3-tech/

Molendijk, M. J., de Putter, F. A. M., Gomony, M. D., Jääskeläinen, P., & Corporaal, H. (2023). BrainTTA: A 28.6 TOPS/W compiler programmable transport-triggered NN SoC. In *2023 IEEE 41st International Conference on Computer Design (ICCD)* (pp. 78–85). IEEE. https://doi.org/10.1109/ICCD58817.2023.00022

Nadalini, A., Rutishauser, G., Burrello, A., Bruschi, N., Garofalo, A., Benini, L., Conti, F., & Rossi, D. (2023). A 3 TOPS/W RISC-V parallel cluster for inference of fine-grain mixed-precision quantized neural networks. In *2023 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)* (pp. 1-6). IEEE. https://doi.org/10.1109/ISVLSI59464.2023.10238679

Nagel, M., Fournarakis, M., Amjad, R. A., Bondarenko, Y., van Baalen, M., & Blankevoort, T. (2021). *A white paper on neural network quantization* (arXiv:2106.08295). arXiv. https://doi.org/10.48550/arXiv.2106.08295

Nguyen, D., Kim, D., & Lee, J. (2017). Double MAC: Doubling the performance of convolutional neural networks on modern FPGAs. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017* (pp. 890-893). IEEE. https://doi.org/10.23919/DATE.2017.7927113

Ozer, E., Kufel, J., Prakash, S., Raisiardali, A., Kindgren, O., Wong, R., Ng, N., Jausseran, D., Alkhalil, F., Kong, D., Hills, G., Price, R., & Reddi, V. J. (2024). Bendable non-silicon RISC-V microprocessor. *Nature, 634*(8033), 341-346. https://doi.org/10.1038/s41586-024-07976-y

Papon, C. (2017). *VexRiscv: A FPGA friendly 32 bit RISC-V CPU implementation* [Computer software]. SpinalHDL. Retrieved November 9, 2026, from https://github.com/SpinalHDL/VexRiscv

Parashar, A., Pellauer, M., Adler, M., Ahsan, B., Crago, N., Lustig, D., Pavlov, V., Zhai, A., Gambhir, M., Jaleel, A., Allmon, R., Rayess, R., Maresh, S., & Emer, J. (2013). Triggered instructions: A control paradigm for spatially-programmed architectures. In *Proceedings of the 40th Annual International Symposium on Computer Architecture (ISCA '13)* (pp. 142–153). ACM. https://doi.org/10.1145/2485922.2485935

Patterson, D. A., & Hennessy, J. L. (2021). *Computer organization and design RISC-V edition: The hardware software interface* (2nd ed.). Morgan Kaufmann.

Perotti, M., Zhang, Y., Cavalcante, M., Mustafa, E., & Benini, L. (2024). MX: Enhancing RISC-V's vector ISA for ultra-low overhead, energy-efficient matrix multiplication. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)* (pp. 1–6). IEEE. https://doi.org/10.23919/DATE58400.2024.10546720

Prakash, S., Callahan, T., Bushagour, J., Banbury, C., Green, A. V., Warden, P., Ansell, T., & Reddi, V. J. (2023). CFU Playground: Full-stack open-source framework for tiny machine learning (TinyML) acceleration on FPGAs. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)* (pp. 157–167). IEEE. https://doi.org/10.1109/ISPASS57527.2023.00024

Rastegari, M., Ordonez, V., Redmon, J., & Farhadi, A. (2016). XNOR-Net: ImageNet classification using binary convolutional neural networks. In B. Leibe, J. Matas, N. Sebe, & M. Welling (Eds.), *Computer Vision - ECCV 2016* (pp. 525-542). Springer. https://doi.org/10.1007/978-3-319-46493-0_32

RISC-V International. (2025). *Vector dot-product extension (Zvqdotq)* (Version 0.0.2) [Draft specification]. GitHub. https://github.com/riscv/riscv-dot-product

RISC-V International. (2026a). *RISC-V packed SIMD "P" extension* (Version 0.6, Developed state) [Draft specification]. GitHub. https://github.com/riscv/riscv-p-spec

RISC-V International. (2026b). *Zvvm integrated matrix extension v0.9.0: ARC review candidate* [Draft specification]. GitHub. https://github.com/riscv/integrated-matrix-extension/releases/tag/v0.9.0

Schuiki, F., Zaruba, F., Hoefler, T., & Benini, L. (2021). Stream semantic registers: A lightweight RISC-V ISA extension achieving full compute utilization in single-issue cores. *IEEE Transactions on Computers, 70*(2), 212-227. https://doi.org/10.1109/TC.2020.2987314

Serafin, N., Ghosh, S., Desai, H., Beckmann, N., & Lucia, B. (2023). Pipestitch: An energy-minimal dataflow architecture with lightweight threads. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '23)* (pp. 1409–1422). ACM. https://doi.org/10.1145/3613424.3614283

Sharify, S., Delmas Lascorz, A., Siu, K., Judd, P., & Moshovos, A. (2018). Loom: Exploiting weight and activation precisions to accelerate convolutional neural networks. In *Proceedings of the 55th Annual Design Automation Conference (DAC '18)* (pp. 1-6). Association for Computing Machinery. https://doi.org/10.1145/3195970.3196072

Sharma, H., Park, J., Suda, N., Lai, L., Chau, B., Kim, J. K., Chandra, V., & Esmaeilzadeh, H. (2018). Bit Fusion: Bit-level dynamically composable architecture for accelerating deep neural network. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)* (pp. 764-775). IEEE. https://doi.org/10.1109/ISCA.2018.00069

Shi, M., Jain, V., Joseph, A., Meijer, M., & Verhelst, M. (2024). BitWave: Exploiting column-based bit-level sparsity for deep learning acceleration. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)* (pp. 732-746). IEEE. https://doi.org/10.1109/HPCA57654.2024.00062

Skillman, A., & Edsö, T. (2020). A technical overview of Cortex-M55 and Ethos-U55: Arm's most capable processors for endpoint AI. In *2020 IEEE Hot Chips 32 Symposium (HCS)* (pp. 1-20). IEEE. https://doi.org/10.1109/HCS49909.2020.9220415

Smith, J. E. (1984). Decoupled access/execute computer architectures. *ACM Transactions on Computer Systems, 2*(4), 289–308. https://doi.org/10.1145/357401.357403

Texas Instruments. (2001a). *TMS320C54x DSP reference set, Volume 1: CPU and peripherals* (Literature No. SPRU131G). https://www.ti.com/lit/pdf/spru131

Texas Instruments. (2001b). *TMS320C54x DSP reference set, Volume 2: Mnemonic instruction set* (Literature No. SPRU172C). https://www.ti.com/lit/pdf/spru172

Tschand, A., Rajan, A. T. R., Idgunji, S., Ghosh, A., Holleman, J., Kiraly, C., Ambalkar, P., Borkar, R., Chukka, R., Cockrell, T., Curtis, O., Fursin, G., Hodak, M., Kassa, H., Lokhmotov, A., Miskovic, D., Pan, Y., Manmathan, M. P., Raymond, L., . . . Reddi, V. J. (2025). MLPerf Power: Benchmarking the energy efficiency of machine learning systems from µWatts to MWatts for sustainable AI. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (pp. 1201-1216). IEEE. https://doi.org/10.1109/HPCA61900.2025.00092

Umuroglu, Y., Fraser, N. J., Gambardella, G., Blott, M., Leong, P., Jahre, M., & Vissers, K. (2017). FINN: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)* (pp. 65-74). Association for Computing Machinery. https://doi.org/10.1145/3020078.3021744

Umuroglu, Y., Rasnayake, L., & Själander, M. (2018). BISMO: A scalable bit-serial matrix multiplication overlay for reconfigurable computing. In *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE. https://doi.org/10.1109/FPL.2018.00059

Wang, X., Goyal, V., Yu, J., Bertacco, V., Boutros, A., Nurvitadhi, E., Augustine, C., Iyer, R., & Das, R. (2021). Compute-capable block RAMs for efficient deep learning acceleration on FPGAs. In *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)* (pp. 88-96). IEEE. https://doi.org/10.1109/FCCM51124.2021.00018

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM, 52*(4), 65-76. https://doi.org/10.1145/1498765.1498785

Wolf, C. X. (2015). *PicoRV32: A size-optimized RISC-V CPU* [Computer software]. YosysHQ. Retrieved November 9, 2026, from https://github.com/YosysHQ/picorv32

Xilinx, Inc. (2018). *7 series DSP48E1 slice user guide* (UG479, v1.10, March 27, 2018). https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1

Xilinx, Inc. (2019). *7 series FPGAs memory resources user guide* (UG473, v1.14, July 3, 2019). https://docs.amd.com/v/u/en-US/ug473_7Series_Memory_Resources

Xilinx, Inc. (2020). *7 series FPGAs data sheet: Overview* (DS180, v2.6.1, September 8, 2020). https://docs.amd.com/v/u/en-US/ds180_7Series_Overview

Yao, Z., Dong, Z., Zheng, Z., Gholami, A., Yu, J., Tan, E., Wang, L., Huang, Q., Wang, Y., Mahoney, M. W., & Keutzer, K. (2021). HAWQ-V3: Dyadic neural network quantization. In M. Meila & T. Zhang (Eds.), *Proceedings of the 38th International Conference on Machine Learning* (Vol. 139, pp. 11875-11886). PMLR.

https://proceedings.mlr.press/v139/yao21a.html

Yiu, J. (2019). *Introduction to Armv8.1-M architecture* [White paper]. Arm. https://www.arm.com/resources/white-paper/intro-armv8-1-m-architecture